VRIJE
UNIVERSITEIT
BRUSSEL

Master thesis submitted in order to be awarded the degree of
Master of Science in Engineering Technology:
Electronics-ICT - networks

RADIO DUTY CYCLING FOR MULTI-HOP 2.4 GHZ LORA

Jutter Peeters

2022-2023

promoter: Prof. Kris Steenhaut, Prof. An Braeken
supervisor: Roald Van Glabbeek

ENGINEERING SCIENCES

Acknowledgements

I would like to extend my sincere thanks to my supervisor Roald Van Glabbeek, for his guidance throughout the year and his support during the difficulties.

I would also like to express my sincere gratitude to my promotor Prof. Kris Steenhaut for her expertise, her feedback and for her support in the last weeks before the deadline.

Lastly I would like to extend my gratitude to my promotor prof. An Braeken for her feedback throughout the year.

Jutter Peeters

Abstract

Low-power long range communication is an important part of the growing IoT scene. A recent development is the availability of LoRa in the 2.4 GHz frequency band.

Originally LoRa operates in the sub-GHz ISM (Industrial, scientific and medical) frequency bands and uses a modulation technology based on the CSS (Chirp Spread Spectrum). This makes it ideal for LPWAN's (Low Power, Wide Area networks). LoRaWAN (Long Range Wide Area Network) defines the communication protocol and architecture that is typically used in conjunction with LoRa.

LoRaWAN uses an ALOHA-like MAC (Medium Access Control) protocol. It uses a star topology where battery operated end devices are connected to a LoRa Gateway connected to the Internet. Devices don't wait for the channel to be clear before sending, but instead wait for an ACK (acknowledgement) of successful receipt.

Now LoRa is also available in the 2.4 GHz ISM band, which offers the possibility of higher data rates at a shorter range and also mitigates the 1% duty cycle restriction on the sub-GHz ISM band.

Research towards more optimized LoRa protocols for certain circumstances is also very active, in particular with respect to multi-hop solutions. Multi-hop solutions might be able to help with bandwidth congestion, improve coverage and offer alternative routes in case of gateway outages, depending on the type of traffic

This thesis contributes to this research by studying RDC (Radio Duty Cycling) protocols for LoRa.

RDC protocols are important to achieve energy efficiency for Node-to-Node communication in multi-hop solutions, The purpose of the RDC protocol is to save energy by turning the radio off as much as possible while no communication is happening, while still providing the possibility of communication if needed.

To evaluate RDC protocols for LoRA 2.4 GHz, I will make use of the Zolertia Firefly boards from the lab that use the Contiki operating system, together with a hardware shield using the sx1280 LoRa transceiver.

A driver using Contiki-NG for the sx128x LoRa transceivers has partially been written by MSc. Thomas Perale [14]. However it still needs to be finished, adapted and tested with our shield and boards.

Finally RDC protocols will be analyzed and one will be designed and implemented for LoRa 2.4 GHz using the Zolertia Firefly with the Contiki-NG operating system and a hardware shield with the sx1280 LoRa transceiver.

Samenvatting

Langeafstandscommunicatie met lage energie consumptie is een belangrijk onderdeel van de groeiende IoT markt. Een recente ontwikkeling in dit domein is de ontwikkeling van LoRa voor de 2.4GHz frequentie band.

Oorspronkelijk maakt LoRa gebruik van de sub-GHz ISM (Industrial, scientific and medical) frequentiebanden en gebruikt het een modulatietechiek gebaseerd op CSS (Chirp Spread Spectrum). Dit maakt het ideaal voor LPWAN's (Low Power, Wide Area Networks). LoRaWAN (Long Range Wide Area Network) is het netwerk protocol dat typisch wordt gebruikt samen met LoRa.

LoRaWAN gebruikt een ALOHA-achtig MAC protocol (Medium Access Control). Het maakt gebruik van een ster topologie waarbij op batterijen werkende apparaten verbonden zijn met een LoRa Gateway die is verbonden met het internet. Hierbij wachten apparaten niet tot het kanaal vrij is voordat ze een bericht versturen, maar wachten ze op een ACK (Acknowledgment) van succesvolle ontvangst.

Nu dat LoRa ook beschikbaar is in de 2.4 GHz band, biedt dit de mogelijkheid aan van hogere datasnelheden op een korter bereik. Het zorgt er ook voor dat er geen rekening meer moet gehouden worden met de 1% duty cycle beperking op de sub-GHz ISM band.

Onderzoek naar meer geoptimaliseerde LoRa protocollen voor bepaalde omstandigheden is ook zeer actief, in het bijzonder multi-hop oplossingen. Multi-hop oplossingen kunnen mogelijk helpen bij bandbreedte congestie, de dekking verbeteren en alternatieve routes aanbieden indien er een gateway uitvalt.

Deze thesis draagt hieraan bij door onderzoek te doen naar RDC (Radio Duty Cycling) protocollen voor LoRa 2.4 GHz.

RDC protocollen zijn belangrijk voor de energie efficiëntie van node-to-node communicatie in multi-hop oplossingen. Het doel van een RDC protocol is om energie te besparen door de radio zoveel mogelijk uit te schakelen terwijl er toch nog de mogelijkheid wordt behouden tot onderlinge communicatie.

Om RDC protocollen voor LoRa 2.4 GHz te evalueren wordt er gebruik gemaakt van de Zolertia Fireflies bordjes uit het lab, die gebruik maken van het Contiki besturing systeem, samen met een hardware shield met de LoRa sx1280 transceiver.

Een driver voor de sx128x transceivers voor Contiki-NG is reeds deels geschreven door MSc.Thomas Perale [14]. Deze driver moest echter wel nog afgewerkt, aangepast en getest worden met de nieuwe hardware.

Tot slot moeten er ook RDC protocollen geïmplementeerd en geanalyseerd worden in Contiki-NG.

Resumé

Les communications longue distance avec consommation d' énergie faible est important pour la scène IoT en pleine croissance. Un développement récent dans ce domaine est le développement de LoRa pour la bande de fréquences 2,4 GHz.

A l'origine LoRa utilise les bandes de fréquences sub-GHz ISM (Industriel, Scientifique et Médical) et utilise une technique de modulation basée sur le CSS (Chirp Spread Spectrum). Cela le rend idéal pour les LPWAN (Low Power, Wide Area Networks). LoRaWAN (Long Range Wide Area Network) est le protocole réseau généralement utilisé avec LoRa.

LoRaWAN utilise un protocole MAC (Medium Access Control) comme ALOHA. Il utilise une topologie en étoile dans laquelle les appareils alimentés par batterie sont connectés à une LoRa gateway connectée à Internet. Les appareils n'attendent pas que le canal soit libre avant d'envoyer un message, mais attendent un ACK (Acknowledgement) de réception réussie.

Récent, LoRa est disponible dans la bande ISM 2.4 GHz, cela offre la possibilité de débits de données plus élevés avec une gamme plus courte. Il élimine également la limitation du duty cycle de 1% sur la bande sub-GHz ISM.

La recherche de protocoles LoRa plus optimisés pour certaines conditions est très active, notamment les solutions multi-hop. Les solutions multi-hop peuvent potentiellement contribuer à réduire la congestion de la bande passante, améliorer la couverture et offrir des routes alternatifs en cas de panne d'une LoRa gateway.

Cette thèse y contribue en recherchant les protocoles RDC (Radio Duty Cycling) pour LoRa 2.4 GHz.

Les protocoles RDC sont importants pour l'efficacité énergétique de la communication node-to-node dans les solutions multi-hop. Le but d'un protocole RDC est d'économiser de l'efficacité énergétique par eteindre la radio autant que possible en conservant la possibilité de communication.

Pour évaluer les protocoles RDC pour LoRa 2.4 GHz, on utilise les Zolertia Fireflies du laboratoire, qui utilise Contiki OS (Operating System), avec un hardware shield contenant la LoRA sx1280 transceiver. Un driver pour les transceivers sx128x pour Contiki-NG est déjà écrit en partie par MSc. Thomas Perale [14]. Cependant, ce driver doit encore être finalisé, modifié et testé avec le nouveau matériel.

Finalement, les protocoles RDC doivent également être implémentés et analysés en Contiki-NG.

Contents

List of Figures

List of Tables

Listings

Acronyms

ACK	Acknowledgment
LPWAN	Low Power Wide Area Network
IoT	Internet of Things
CSS	Chirp Spread Spectrum
LoRa	Long Range
MAC	Medium Access Control
ISM	Industrial Scientific and Medical
RDC	Radio Duty Cycling
PHY	Physical
OSI	Open Systems Interconnection
DSSS	Direct Sequence Spread Spectrum
SNR	Signal to Noise Ratio
CCA	Clear Channel Assessment
CAD	Clear Activity Detection
RSSI	Received Signal Strength Indicator
IPS	Internet Protocol Suite
SPI	Serial Peripheral Interface
SDR	Software Defined Radio
IRQ	Interrupt Request
DIO	Digital Input Output
FS	Frequency Synthesis
CR	Coding Rate
CRC	Cyclic Redundancy check
VM	Virtual Machine
CS	Chip Select
SF	Spreading Factor
CR	Coding Rate
VM	Virtual Machine
CRC	Cyclic Redundancy Check
NU	Not Used
MDK	Microcontroller Development Kit

Chapter 1

Introduction

The IoT (Internet of Things) led to the deployment of many applications that provide smart services to users. Examples of such applications are home automation, smart agriculture, smart vehicles etc. Today, billions of IoT devices are deployed around the world and the number keeps on growing.

Wireless devices play a large role in the development of the IoT scene. For these devices it was important that they have a low cost, are energy efficient and can cover large areas. This led to the development of the LPWANs (LPWANs).

One such LPWAN technology is LoRa (Long Range). LoRa can be seperated in the LoRa physical layer and into LoRaWAN MAC (Medium Access Control) protocol.

The physical layer specifies the modulation techniques used for LoRa like the CSS (Chirp Spread Spectrum) modulation technique which offers good resilience to interference and to doppler-effect [15]. It uses the unlicensed sub-GHz ISM (Industrial Scientific and Medical) bands. Those frequency bands have a few drawbacks. The LoRa hardware needs to work for different frequency bands in different parts of the world. For example the 868 MHz band is used in Europe while the 915 MHz band is used in North America and the 433 MHz band is used in Asia. There is also a duty cycle restriction, which dictates that a transmitter can only transmit 1% of the time.

To combat those issues a LoRa transceiver operating at the globally available 2.4GHz ISM band was developed by Semtech, the founding member of the LoRa Alliance. Since those LoRa devices operate at a higher frequency, they are also able to transmit at higher data rates. There is also no duty cycle restriction, which means that devices are able to send way more data. This opens a possible LoRa implementation for a whole new array of applications.

LoRa 2.4 GHz may be the ideal solution for devices that need a longer range than classic 2.4 GHz technologies such as Wi-Fi and Bluetooth. But also need a higher data rate than standard LPWAN's.

LoRaWAN, the MAC protocol most often used in conjuction with LoRa specifies an ALOHA-based MAC protocol. LoRaWAN makes use of a one-hop star topology where battery operated end devices are connected to a LoRa gateway connected to the Internet. Devices do not check if the channel is clear before transmitting, as the signal level can go lower than the noise level. After having sent a message, nodes stay awake during a while in listen mode to wait for an ACK (acknowledgement) of successful receipt.

Although this single-hop ALOHA-like strategy is convenient from a business and an ease of use perspective, it also has some drawbacks like scalability, coverage, data rate, gateway outage.

LoRaWAN which uses an ALOHA like protocol may not perform well with dense networks and high traffic loads because of collisions [3]. LoRaWAN may also have trouble covering large areas with few gateways or in areas where the path loss is high. Blind spots can appear. This is where a multi-hop protocol may offer solutions. A multi-hop protocol may improve coverage, can help with bandwidth congestion, and offer alternative routes in case of gateway outages.

Unfortunately, a multi-hop implementation has a higher complexity than the LoRaWAN one-hop ALOHA-like protocol. There is no time-synchronization needed for the LoRaWAN-based nodes since they can send when they want as the gateway is always listening. In a multi-hop solution however, a node could have to send to a neighbour node, instead of to the always awake radio of the base station. Therefore, those nodes will have to agree upon when and on which channel to communicate and when to sleep. Indeed, an efficient RDC (Radio Duty Cycling) protocol will be very important, since the devices are most often energy constrained. RDC protocols have as function to save energy by turning the radio of the device off as much as possible without losing the possibility of communication with their neighbours when needed.

The control messages that are needed to achieve a multi-hop solution will create extra overhead and increase energy consumption compared to a single-hop solution. Devices may however also be able to save some energy since they will be able to send messages to closer neighbours instead of the farther away gateways. Doing so they will be able to reduce their spreading factor and reduce as such their Time On Air and as a consequence time during which the radio stays active.

This thesis contributes to research towards RDC protocols for LoRa 2.4 GHz. To achieve this I will use the Zolertia Firefly boards from the lab at the Vrije Universiteit Brussel that use Contiki-NG as an operating system, together with a hardware shield containing the sx1280 LoRa 2.4 GHz transceiver from Semtech.

As a first endeavor, the programming of the driver for the sx128x chips for contiki-ng, is done, tested and evaluated. This work is based on a preliminary version of the driver made for the SX1280 chip written by MSc. Thomas Perale [14]. This driver has been adapted and enhanced to work with our hardware. Afterwards, an investigation of possible RDC protocols is done. An RDC protocol is chosen inspired by ContikiMAC and a first implementation has been done. This may greatly reduce the energy consumption for LoRa multi-hop solutions.

1.1 Objectives

The objectives of this thesis can be outlined as follows:

- study LoRa and LoRa 2.4 GHz
- get more familiar with Contiki-ng and the driver [14] for the sx128x transceivers
- make a hardware shield containing the sx1280 transceiver for the Zolertia firefly
- test and evaluate the driver and enhance and debug where needed
- Choose/design, implement, test and evaluate a preliminary version of an RDC protocol for LoRa 2.4 GHz

1.2 Structure of the manuscript

The structure can be summarised as follows:

- chapter 1, this chapter, gives an introduction to this work and an outline of the objectives and the structure of the thesis
- chapter 2 will introduce the topics related to this work
- chapter 3 will provide an introduction to the hardware used in this work

- chapter 4 will give an overview of the driver and the challenges faced getting the hardware and the driver to work
- chapter 5 will give an overview of the RDC protocol implemented
- chapter 6 will conclude the thesis and discuss possible future work

2.1 Introduction

In this chapter, background information is given on topics and technologies related to this thesis.

The first topic presented in this chapter is LoRa. This topic is explained in section 2.2 Both the physical layer and the MAC protocol LoRaWAN will be presented, as well as LoRa 2.4 GHz.

The second topic presented is the lightweight operating system Contiki-NG. This topic is handled in section 2.3. An overview of the structure, the configuration system, processes and events and the radio structure is given in this section.

The last topic presented will be Radio Duty Cycling and in particular, the three main RDC protocols implemented in Contiki-OS. These RDC protcols are ContikiMAC, X-MAC and LPP. This topic is explained in detaill in section 2.4.

2.2 LoRa

LoRa (Long Range) is a spread spectrum modulation technique derived from CSS (Chirp Spread Spectrum) technology developed by Semtech. It is developed for LPWANs (Low Power Wide Area Networks) for which range and energy consumption are of paramount importance.

The name LoRa is used to refer to two distinct layers. LoRa PHY (Physical) the physical layer that is developed by Semtech known for its CSS modulation technique and LoRaWAN (Long Range Wide Area Network), a MAC layer protocol most oftenly used in conjunction with LoRa physical layer developed by the LoRa-alliance, a non-profit technology organisation of which Semtech is a founding member and sponsor.

The OSI (Open Systems Interconnection) seven-layer network model is depicted in figure 2.1 and the respective LoRa layers are situated in this model.

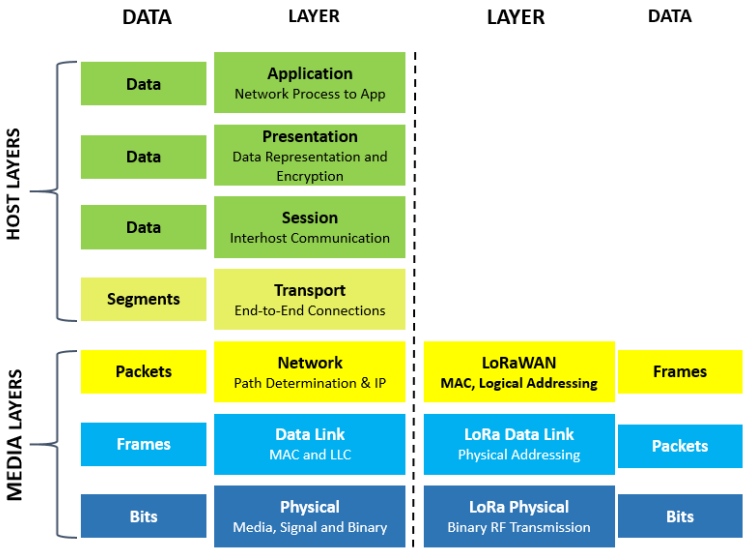


Figure 2.1: OSI seven-layer network model, from [17]

Figure 2.2 shows which gap LoRa fills in the collection of wireless technologies. For some applications where a high data rate is needed, like video-surveillance, Wi-Fi would be a much better option than LoRa. However, for applications with lower data rates where long range and power consumption are the most important factors, like smart-agriculture, LoRa would be a better choice [9], [18].

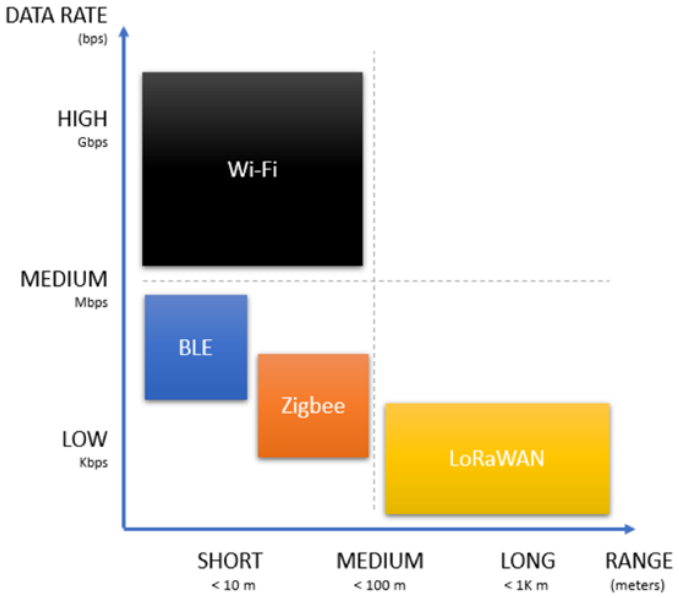


Figure 2.2: Range versus data rate of various wireless technologies, from [9]

2.2.1 LoRa PHY

LoRa PHY (Physical) is a proprietary spread-spectrum modulation technique derived from existing CSS (Chirp Spread Spectrum) technology. It operates at the sub-GHz ISM (Industrial Scientific and Medical) bands, but is now also available for the 2.4 GHz ISM band.

While Direct Sequence Spread Spectrum (DSSS) techniques makes use of orthogonal Spreading codes to spread the signal, LoRa makes use of orthogonal SFs (Spreading Factors). A spreading factor can be between 5-12 and a higher SF means a longer range but also a lower data rate and a longer time on air.

Because of the SFs a trade-off between data-rate and range is possible. A device closer to a gateway (which is connected to the Internet), can use a lower spreading factor than a device further away. A device that is far from the gateway will use a higher processing gain and reception sensitivity. This will increase the range, however the data rate will be lower [17].

We will now take a closer look to what a spread-spectrum modulation technique exactly means in section 2.2.1.1 and after the characteristics of LoRa modulation will be explained in section 2.2.1.2.

2.2.1.1 DSSS and LoRa Spread Spectrum

In figure 2.3, traditional DSSS is shown. When the data signal is multiplied with a spreading code, that has a much faster bit rate than the original signal, a signal with much higher frequency components is created. The signal is spread out over the frequency domain. The bits of the spreading code are called chips, this to differentiate them from the bits from the original signal. The signal can be restored at the receiver by multiplying the received signal with the spreading code again. The longer the Spreading code, the higher the noise resilience, but the lower the transmission speed.

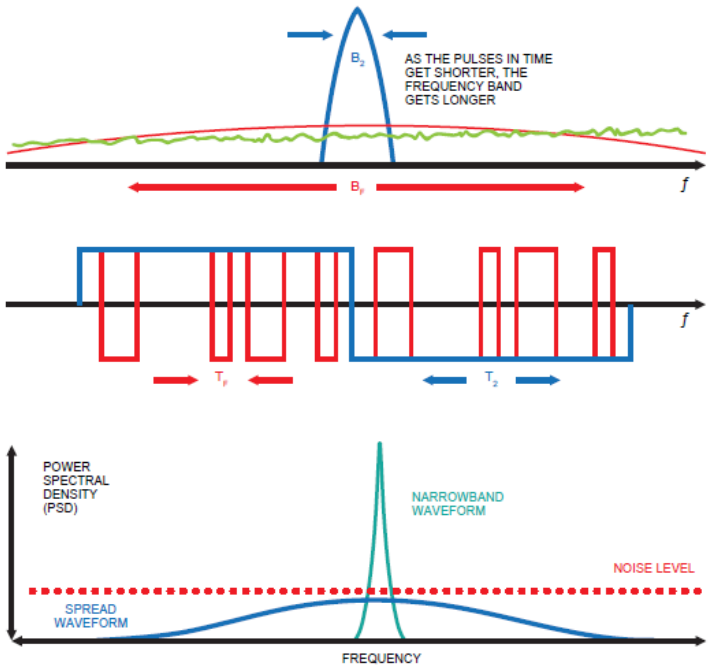


Figure 2.3: DSSS system carrier phase transmitter signal changes, from [17]

Spreading codes are not all equal. "Good" patterns have precise properties. They should have equal probabilities for 0 and 1, good auto-correlation properties (the pattern correlated with

itself must only show one peak), and correlation of the distinct patterns to represent a 1 or a 0 should not show any significant peak, or they might be misunderstood for each other [11].

DSSS has a high noise resistance which is illustrated in the following figures. In figure 2.4 we can see a signal with sequence "101" in gray and a spreading code in green. The spreading code is a "Barker sequence" which means that the green pattern is used for a "1" and the negated version is used for a "0". This leads to the spread signal in red.

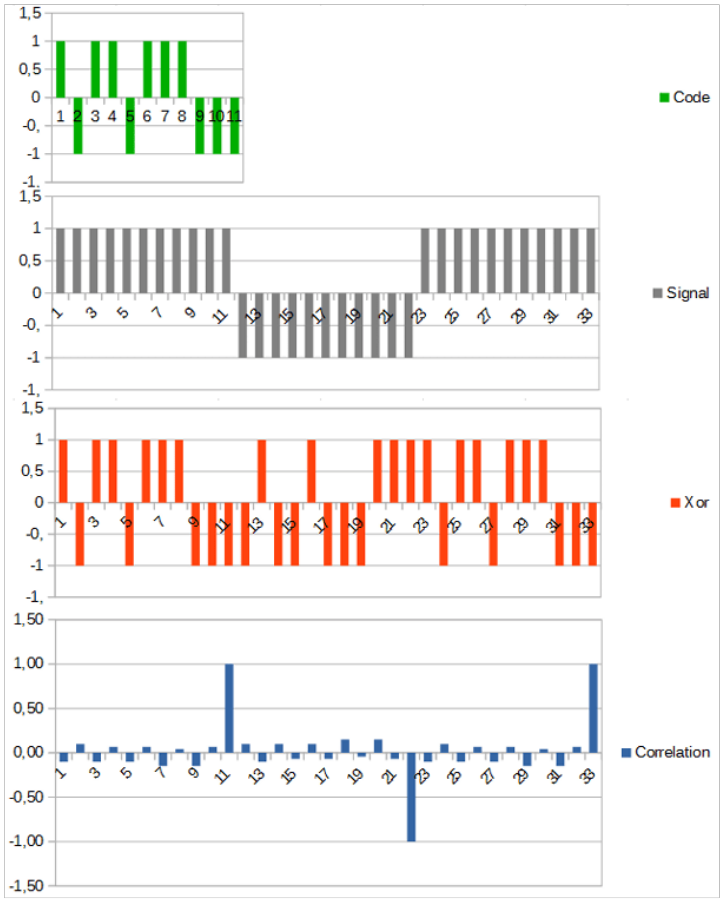


Figure 2.4: An illustration of a simple DSSS encoding and decoding, using a 11-bit Barker-code., from [11]

At the receiver this signal can be reconstructed by the correlation between the received signal and the spreading code.

In figure 2.5, there is random noise added with a value between "-1" and "+1" to the received signal in red. The noise has the same amplitude as the original signal, but when the correlation between this received signal and the spreading code is taken, it can be observed that we are still able to reconstruct the signal.

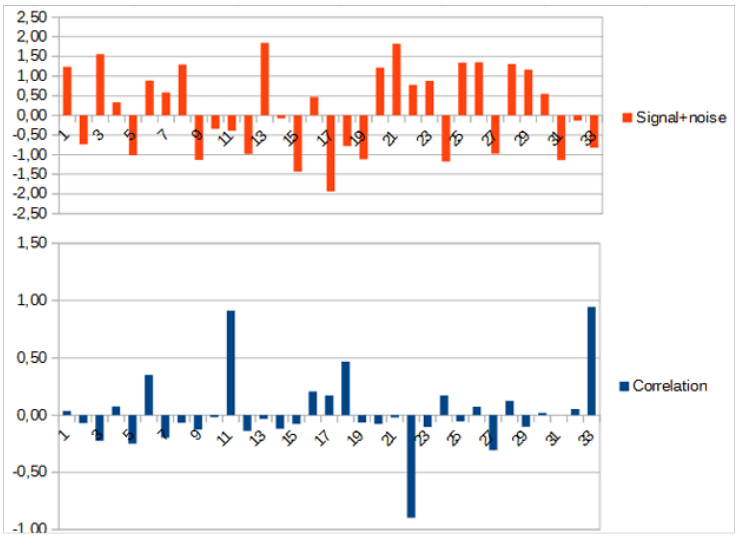


Figure 2.5: signal from figure 2.4 with high level of noise and reconstruction, from [11]

This can be explained as follows.

$$(signal+noise)*(spreadingcode) = (signal*spreadingcode)+(noise*spreadingcode) \quad (2.1)$$

The last term of the equation will be close to zero, since the correlation between the random noise and the spreading code will be very low compared to the correlation between the received signal and the spreading code.

This means that even signals under the noise floor can be demodulated.

DSSS also is very resistant against narrow-band interference. This is because the signal is spread over a wide range of frequencies and the signal can most often be reconstructed after demodulation despite narrow-band interference . This is also shown in figure 2.6

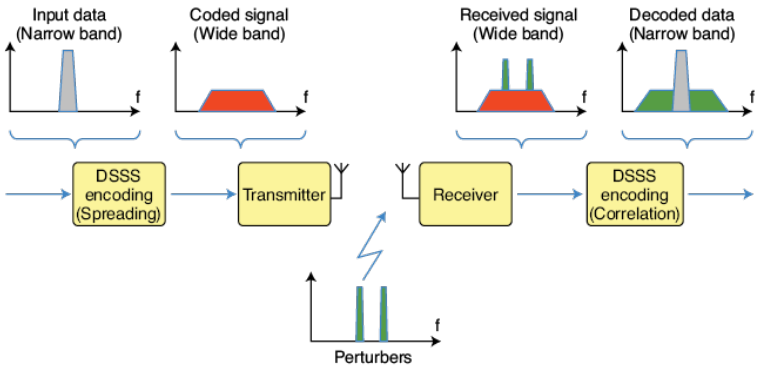


Figure 2.6: Perturbers, which are narrow band, have a low impact on the result of the correlation, from [11]

It is also important to note that many DSSS signals can be sent at the same time. The only requirement is that they use different spreading codes. Those spreading codes will need to be as orthogonal as possible which means that they must have low cross-correlation. This way messages from one device will not interfere with the messages from other devices, as those will be regarded as noise. This can be compared to the last term of equation 2.1.

Another way to explain this is by the Shannon-Hartley Theorem. The theorem states the maximum rate at which information can be transmitted over a channel of a certain bandwidth on which white noise is superposed.

$$C = B * log_2(1 + S/N) \tag{2.2}$$

C = channel capacity (bit/s) B = channel bandwidth (Hz) S = average received signal power N = average noise

If the SNR (Signal to Noise Ratio) is small and $S/N << 1$ applies. The second term can be approximated.

$$log_2(1 + S/N) = 1/ln_2 * ln(1 + S/N) \approx 1/ln_2 * S/N \approx 1.44 * S/N \tag{2.3}$$

This means that $C/B \approx 1.44 * S/N$. From this it is visible that to be able to send data at a certain data rate and with a known SNR, only the bandwidth needs to be increased. This is what DSSS does.

One of the downsides of DSSS is that it requires a highly-accurate and expensive clock. In addition it may also be quite energy intensive for power-constrained devices[17] [16].

This is solved in LoRa by using CSS technology. LoRa uses a chirp as the basis for its modulation. A chirp is a sinusoidal wave form that increases or decreases linearly in time. This is called an up chirp or a down chirp. LoRa uses multiple symbols. In total there are 2^{SF} symbols where SF is the spreading factor, a number between 7 and 12.

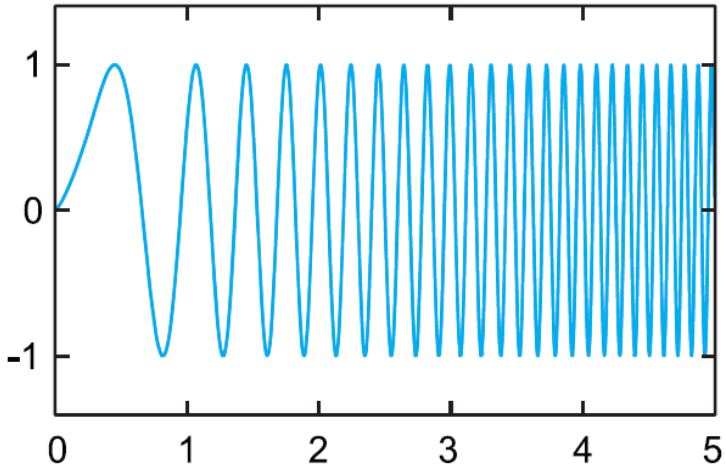


Figure 2.7: LoRa Chirp Spread Spectrum illustratiom, from [17]

There are 2 main problems for DSSS, the synchronization of the symbols and the computational burdensome correlations that are needed for each symbol. Lora uses a lot of symbols, namely 2^{SF} symbols. This means that a lot of correlation calculations have to be made by the receiver.

The first problem is tackled by always sending a preamble of multiple base chirps. By receiving this preamble the receiver is able to align his frames with those sent by the transceiver, synchronizing the receiver.

The second problem is more complex. LoRa uses a mathematical trick to easily demodulate the symbols instead of doing 2^{SF} correlations. This trick is very well explained in the following video ([8]) based on [24].

To demodulate the received signal (first term), we need to correlate it with each complex conjugate of the symbols (second term). This is visible in equation 2.4.

$$\sum_{k=0}^{2^{SF}-1} r(nT_s + kT) * C^*(nT_s + kT) \tag{2.4}$$

The mathematical trick used in LoRa lays in the second term.

$$C^*(nT_s + kT) = 1/\sqrt{2^{SF}} e^{-j2\pi(s(nT_s)+k*mod2^{SF})k/2^{SF}} \tag{2.5}$$

If we add $+k - k$ in the exponent in equation 2.5, this can be rewritten as in 2.6

$$C^*(nT_s + kT) = 1/\sqrt{2^{SF}} (e^{-j2\pi k^2/2^{SF}})(e^{-j2\pi(s(nT_s)+k*mod2^{SF}-k)k/2^{SF}}) \tag{2.6}$$

In 2.6 the $e^{-j2\pi k^2/2^{SF}}$ part is a down chirp and the $e^{-j2\pi(s(nT_s)+k*mod2^{SF}-k)k/2^{SF}}$ part is a sinusoidal wave form with frequency s.

This means the signal can be reconstructed my multiplying the received signal with the down chirp and performing the correlation with the sinusoidal waveforms corresponding with each symbol with frequency s.

This can be simplified by performing the Fourier transform since we are looking for the frequency s. After the signal is dechirped, the Fourier transform can be used to get a frequency corresponding to a symbol.

2.2.1.2 LoRa Characteristics

The SF spreading factor in LoRa ranges from 5-12. A higher spreading factor means a higher energy consumption, a longer range and a better SNR. However it also increases the time on air. LoRa uses symbols as its basis for communication and in total there are 2^{SF} symbols used. This also means that each symbol consists of 2^{SF} chirps. This to be able to differentiate them. Those SFs are quasi orthogonal which means that they will almost not interfere with each other if multiple devices are transmitting at different SFs in the same channel. The following equation 2.7 for the symbol rate can be found in the sx1280 datasheet [20].

$$R_c = 2^{SF} * R_s \tag{2.7}$$

In equation 2.7, R_c is the chip rate and R_s is the symbol rate.

The symbol period can be given by equation 2.8 [20]. Here stands BW for bandwidth and it can be set as 203, 406, 812 or 1625 kHz

$$T_s = 2^{SF}/BW \tag{2.8}$$

The data rate (without error correction) can be calculated with equation [20].

$$R_b = SF/T_s \tag{2.9}$$

LoRa normally uses error correction. This is defined by the coding rate. The coding rate is either 4/5, 4/6, 4/7 or 4/8. 4/5 means that for each 4 symbols, 1 error correction symbol is sent. A higher CR means better error protection at the cost of a longer ToA (Time on Air). The effective bit rate can then be calculated with equation 2.10 [20].

$$R_{beff} = R_b * 4/(4 + CR) \tag{2.10}$$

2.2.2 LoRaWAN

LoRaWAN is a networking protocol that works in conjunction with LoRa created by the LoRa Alliance. An example of a typical LoRaWAN network can be seen in figure 2.8

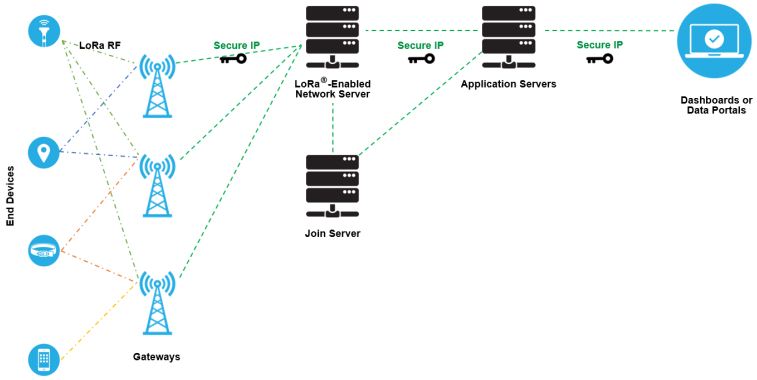


Figure 2.8: A typical LoRaWAN network, from [17]

A LoRaWAN network is typically a star-of-stars topology. The topology consists of end-devices, gateways, a network server, an application server and a join server [6].

End devices are most often energy constrained and transmit data to the gateway. The gateways receive those packets and forward them to the network server. The network server is responsible for managing the MAC layer and verifying the messages and answering to the devices.

The application server receives the messages from the network server and forwards the packets to the associated applications. The join server takes care of the authentication of the end-devices,

There are different classes of end devices [6].

- Class A: this device can send packets at any time. After a transmission they open a reception window to receive an ACK of successful reception.
- Class B: this device can open more reception windows than in class A. The device will periodically open a reception window to receive messages from the gateway. The gateway and devices uses beacon messages for time-synchronization.
- Class C: Opens two reception windows and keeps the second one open until the next uplink transmission. These are typically non energy constrained devices.

LoRaWAN uses an ALOHA-like protocol. Which means that end-devices will not check if a channel is free before sending a message, but wait for an ACK (acknowledgment) of successful reception. Messages that are send in the same channel with the same SF can cause collisions. This means that LoRAWAN might not be very effective in very dense networks [3].

For LoRa 2.4 GHz there is no duty cycle restriction and this opens the door to applications that may transmit much more often, which can make the problem of packet collisions more apparent.

This problem might be solved by implementing RDC protocols.

LoRaWAN uses a single-hop technology, however a multi-hop solution may have multiple advantages in certain scenarios. It may help with the collision problem since less devices will have to be connected directly to the gateway. LoRaWAN may also have trouble covering large areas with few gateways or in areas where the path loss is high. Blind spots can appear. In case of a gateway outage, a multi-hop solution might also be able to offer alternate routes to another gateway.

For this an efficient RDC protocol will be needed. End-devices have to be able to communicate with each other and their radio can't always be turned on since they are energy constrained. It may also be more energy efficient since devices won't have to send their messages to a far away gateway but can send them instead to a neighboring device. However the synchronization of the devices will also increase the overhead which will also increase the energy consumption. Each technology has its own advantages and disadvantages.

2.2.3 LoRa 2.4 GHz

With the move from the sub-GHz bands, manufacturers are able to design a uniform LoRa chip that functions independently of the region of deployment [10]. This is because in different parts of the world, different sub-GHz ISM bands are used. It also solves the problem of the duty cycle restriction of the sub-GHZ ISM bands.

Compared to LoRa in the sub-GHZ band, the higher frequency means that the range is significantly reduced, but it is also possible to use a much higher data-rate. Especially since the maximum bandwidth also increased from 500 kHz to 1600 kHz.

However compared to other technologies operating in the 2.4 GHz ISM band, LoRa still offers a much longer range.

The possibility to increase or decrease the spreading factor and changing the channel bandwidth makes LoRa a very flexible option. This allows a trade-off between range and data rate [10].

The trade off can be seen in the following figures. The maximum range in a free space is around 133 km, in an indoor environment around 74 m and in an urban environment 443 m. This is significantly less than LoRa in the sub-GHZ band, however much higher data rates are possible in the 2.4 GHz version [10].

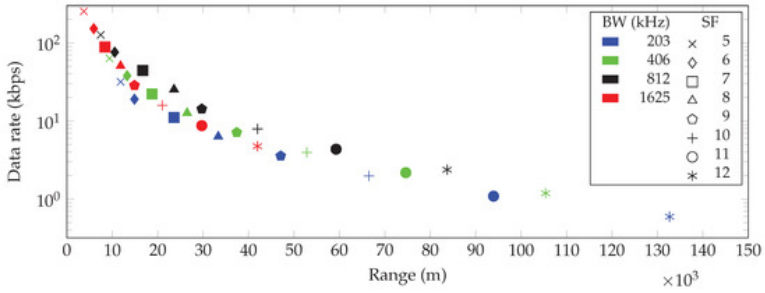


Figure 2.9: Communication range and data rate for every combination of spreading factor (SF) and bandwidth (BW) in a free space line of sight (LoS) environment, from [10]

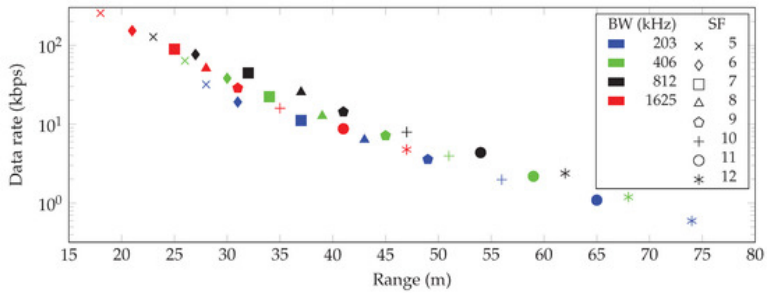


Figure 2.10: Communication range and data rate for every combination of spreading factor (SF) and bandwidth (BW) in an indoor environment, from [10]

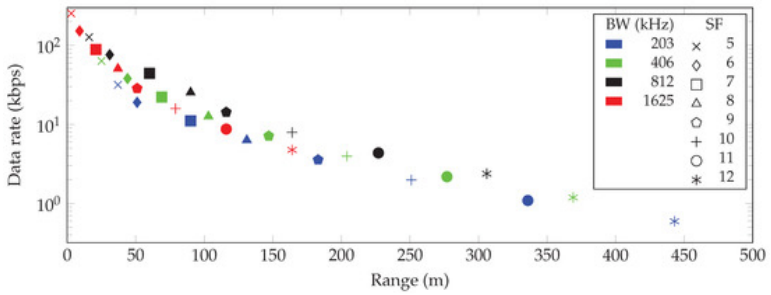


Figure 2.11: Communication range and data rate for every combination of spreading factor (SF) and bandwidth (BW) in an urban environment, from [10]

2.3 Contiki-NG

Contiki is an operating system created in 2002 focused on resource-constrained devices in the IoT. These are devices that are constrained in memory, power and bandwidth. It provides a built-in IPS (Internet Protocol Suite), yet needs very little resources.

A new branch has been developed in 2017 for the next generation IoT devices called Contiki-NG where NG stands for "New Generation".

The focus of Contiki-NG is on reliable and secure IPV6 communication for modern IoT platforms. It also aims to modernize the structure of the operating system and improve the documentation. The code footprint is in the order of a 100 kB and the memory usage can be as low as 10 kB [1]. Contiki-NG also offers a simulation tool called the COOJA simulator, written in Java.

Contiki is used in numerous systems such as sound monitoring, street lights, electrical power meters, industrial monitoring, radiation monitoring, alarm systems, and so on [13].

The driver used for LoRa 2.4 GHz [14] is build on contiki-NG.

We will now go a little deeper into the structure of Contiki-NG, but for more information, I would like to refer to their documentation page [2]. The following explanations and code come from this source.

2.3.1 Repository structure

The Contiki-NG repository structure can be summarized as follows:

- OS: this contains the actual operating system and contains processes, timers, the networking stack, and libraries and services.

- arch: this contains all the hardware-dependent code.
- examples: contains documented example projects
- tools: this contains tools like the COOJA simulator and more.
- tests: this contains all continuous integration tests run for every pull request and merge.

2.3.2 Contiki-NG configuration system

Most of the configuration is done via .h files. However, some configuration is done via the project Makefile.

Modules, that are sub-directories of the OS directory, the MAC layer and the NET layer, can be implemented via the Makefile of a project.

Modules can be implemented by writing "MODULES = os/..." while the MAC layer and NET layer can be implemented by writing "MAKE_MAC_NAME" or "MAKE_NET_NAME" where the "NAME" is to be changed by the name of the MAC or NET layer's name.

For the MAC layer these names can be the following:

- NULLMAC: this MAC layer does nothing
- CSMA: this is the default MAC layer and implements Carrier Sense Multiple Access and uses always-on radios.
- TSCH: Time slotted Channel Hopping. This MAC layer is synchronised, scheduled and uses frequency-hopping.
- BLE: an experimental MAC layer for a Bluetooth radio.
- OTHER: this is used if you have implemented your own custom MAC.

For the NET layer we can either use "NULLNET", which is a NET layer that does nothing, or we can use "IPV6" which is the default and uses the low-power IPV6 stack with 6LoWPAN and RPL. Either RPL Lite, RPL classic or no routing can be used.

At last we can also use "OTHER", which is mainly used in case of a custom NET layer.

All other configuration are made inside the project-conf.h file inside the project directory.

A common list of flags used for projects is given in "os/contiki-default-conf.h".

2.3.3 Processes and events

Projects in Contiki-NG make typically use of the Process abstraction.

A process is declared at the top of a source file. It takes two arguments, the identifier and the name. The second states that the process should be automatically started. Alternatively, we can use the function "process_start()".

```
1    PROCESS(identifier,"name");
2    AUTOSTART_PROCESSES(&identifier);
```

The process itself looks like this:

```
1    PROCESS_THREAD(identifier, ev, data){
2        PROCESS_BEGIN();
3        printf("Hello, world!\n");
4        PROCESS_END();
5    }
```

The "ev" and "data" arguments contain the value of an incoming event and an optional pointer to an event argument object [2].

Contiki is event-based. This means that events will trigger certain processes. Such events can be timers and incoming packets.

An example of such an event is:

```
1   PROCESS_THREAD(test, ev, data){
2       static struct etimer et;
3       PROCESS_BEGIN();
4       etimer_set(&et, CLOCK_SECOND); /* Trigger a timer after 1 second. */
5       while(1) {
6           PROCESS_WAIT_EVENT_UNTIL(etimer_expired(&et));
7           etimer_reset(&et);
8       }
9       PROCESS_END();
10  }
```

In this example a timer is used, and each time we are waiting for the event, the control is yielded back to the scheduler. If the process triggers, the code after the event is executed, and the timer is reset.

It is also possible to write "PROCESS_PAUSE" inside a process. With this, the scheduler will first deliver any queued events and after that schedule the paused process.

It is also possible to yield a process by writing "PROCESS_YIELD". However this means that the scheduler will not schedule this code anymore. Instead it will wait for an incoming event, but without a condition argument. However the process can be scheduled again by another process that calls "process_poll(&identifier)", where the identifier is the identifier of the process that yielded.

Processes can be stopped in three ways. It stops when the "PROCESS_END()" statement is reached and executed, when PROCESS_EXIT() is called and executed or when another process stops the process by calling "process_exit(&identifier)", where the identifier is the identifier of the process that needs to be stopped.

The system-defined events can be seen in table 2.1.

EVENT	**id**	**Description**
PROCESS_EVENT_NONE	0x80	No event.
PROCESS_EVENT_INIT	0x81	Delivered to a process when it is started.
PROCESS_EVENT_POLL	0x82	Delivered to a process being polled.
PROCESS_EVENT_EXIT	0x83	Delivered to an exiting a process.
PROCESS_EVENT_SERVICE_REMOVED	0X84	Unused.
PROCESS_EVENT_CONTINUE	0x85	Delivered to a paused process when resuming execution.
PROCESS_EVENT_MSG	0x86	Delivered to a process upon a sensor event.
PROCESS_EVENT_EXITED	0x87	Delivered to all processes about an exited process.
PROCESS_EVENT_TIMER	0x88	Delivered to a process when one of its timers expired.
PROCESS_EVENT_COM	0x89	Unused.
PROCESS_EVENT_MAX	0x8A	The maximum number of the system-defined events.

Table 2.1: Contiki-NG system-defined events, from [2]

2.3.4 Contiki-NG radio.h

Radio Drivers in Contiki-NG are based on the file "radio.h". This file can be found in the Contiki-NG directory under Contiki-NG/os/dev/radio.h. In this file the structure required for

radio-drivers using Contiki-NG can be found. The driver needs to implement this structure to make use of the Contiki-NG network stack. The structure consists of 14 functions.

- **int init(void):** This function initializes the radio driver and returns 1 if successful.
- **int prepare(payload,payload_len):** This function prepares the radio with a packet to be sent and returns 0 if the packet is copied successful.
- **int transmit(transmit_len):** This function sends the packet that previously has been prepared.
- **int send(payload,payload_len):** This function will behave as a call to prepare immediately followed by a call to transmit.
- **int read(buf,buf_len):** This funtion reads a received packet into a buffer and returns 0 if no packet has been correctly received.
- **int channel_clear(void):** This function performs a CCA and returns one if the channel is clear.
- **int receiving_packet(void):** This function checks if the driver is currently receiving a packet and returns 1 if that is the case.
- **int pending_packet(void):** This function checks if a packet has been received and is available in the buffer and returns 1 if one or more packets are available.
- **int on(void):** This function turns the radio on and returns one if successful.
- **int off(void):** This function turns the radio off and returns one if successful.
- **radio_result get_value(radio_parameter, copied_value):** This function copies the current value of the parameter into copied_value and returns an enumerator of radio_result
- **radio_result set_value(radio_parameter, copied_value):** Comparable to get value, but instead changes the value of the radio parameter.
- At last, there is also get_object() and set_object that are quite similar to the previous 2 functions. The first gets a radio parameter object and the second sets one.

For a more complete explanation of the structure, I would like to refer to the radio.h file found in [1].

2.4 RDC

RDC stands for Radio Duty Cycling and has as purpose to save energy by turning the radio off as much as possible while no communication is happening, while still providing the possibility of communication when needed. For example when the devices are time-synchronized, a schedule can be set up for communication.

The RDC protocols are part of the MAC layer. However in the original Contiki, which I will be referring to as Contiki-OS, the RDC layer is separated from the MAC layer and moved in its own layer called the RDC layer [5]. This layer is situated between the physical layer and the MAC layer. In Contiki-NG however, the RDC layer is removed again and moved back into the MAC layer. This means that in Contiki-NG there is no built-in option to switch to different RDC protocols. Since the LoRa driver [14] is build for Contiki-NG, this will pose some problems.

The three duty cycling mechanisms originally implemented in Contiki-OS are ContikiMAC, X-MAC and LPP (Low Power Probing). There is a fourth one called NullRDC. This RDC protocol leaves the radio always on, and is often used for testing or comparisons.

ContikiMAC and X-MAC use a technique called Low-power listening[5]. This is a technique where receivers periodically turn on their radios to check for radio activity. This is called a CAD (Channel Activity Detection) or a CCA (Clear Channel Assessment). If activity is detected, the radio can be kept on and a message can be received.

The sender will send a number of strobe packets to let the receiver know that it wants to send a message. When the receiver periodically wakes up and detects such a probe, they can let the sender know that they are ready to receive a message. For this, the sender has to turn their radio on for a certain time after sending a probe.

It is possible that the sender puts the intended receiver's address in the strobe so that possible receivers can ignore this probe if it is not intended for them.

The 3 RDC protocols will be further explained in section 2.4.1, 2.4.2 and 2.4.3.

2.4.1 ContikiMAC

A few times per second the receiver wakes up to perform 2 consecutive CCAs. Those CCAs return a RSSI (Received Signal Strenght Indicator). The RSSI is most oftenly measured in decibel-milliwatts ranging from 0 to -120. This means that the closer the value is to 0, the greater the signal strength. If the value of the RSSI is above a certain threshold, activity is detected and the radio can be kept on.

The transmitter has to send messages repeatedly during a period that exceeds the interval between wake-ups of the receiver. When the receiver finally turns its radio on and detects a message, it keeps its radio on. After receiving the complete message it sends an ACK to the sender and returns back to sleep.

We will now take a closer look at the timings needed as shown in figure 2.12.

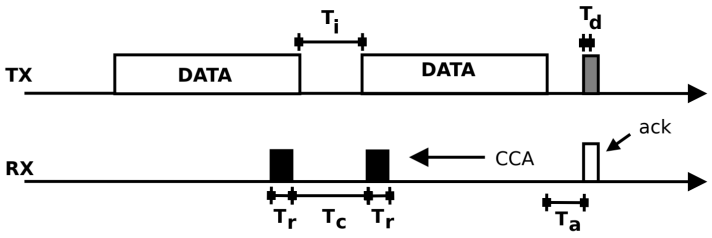


Figure 2.12: Timing constraints in ContikiMAC, from [12], inpired by [7]

- T_i is the interval between two transmissions
- T_a is the time needed to send an ACK after receiving the message
- T_c is the time between two CCA's
- T_r is the time required for a RSSI
- T_d is the time required to detect an ACK from the receiver

– T_s is the required to send a payload packet

The first constraint is that the interval between two data transmissions T_i must be smaller than T_c. This is to ensure that either the first or the second CCA is able to detect the packet. This also sets a requirement for the minimum length of a packet, since the time to send the data packet cannot be so short that it falls between the two CCAs. The transmission time of the shortest packet must be larger than $2T_r + T_c$ [7].

When a packet is detected, the radio is kept on until the full packet is received. After that an ACK is send from the receiver to the sender. This means that the time it takes to send the ACK T_a and the time needed to detect the ACK T_d needs to be shorter than the time between two data packets T_i. This means that $T_a + T_d < T_i$

The constraints can be summarized with equation 2.11.

$$T_a + T_d < T_i < T_c < T_c + 2T_r < T_s \tag{2.11}$$

Contikimac has two additional mechanisms to improve its performance [12]. A phase-lock mechanism and a collision avoidance mechanism.

The phase-lock mechanism allows a device to learn the wake-up schedule of neighboring devices. Because of this, a device is able to predict when neighboring devices will wake up and can start sending just before that. This means that the time and energy used to repeatedly send those messages can be significantly reduced [12] [7].

An added benefit is that because less packages are send, the channel utilization is lowered and the risk of collision is lowered. This mechanism can be seen in figure 2.13.

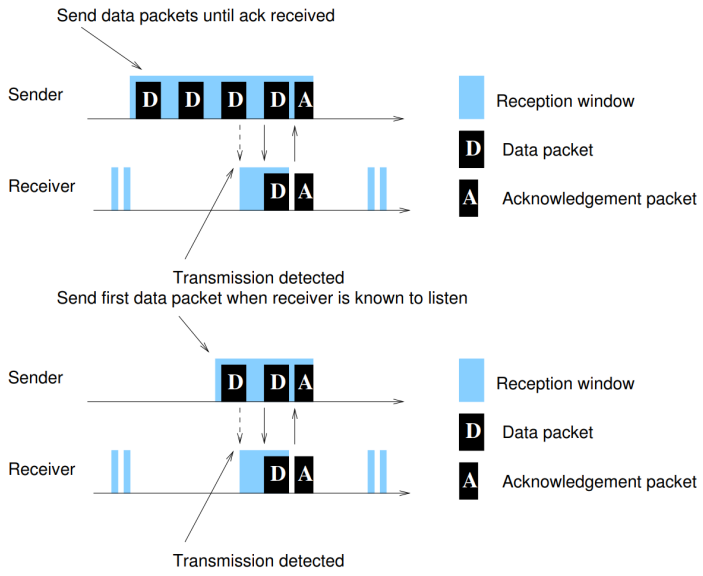


Figure 2.13: Transmission phase-lock: after a successful transmission, the sender has learned the wake-up phase of the receiver and subsequently needs to send fewer transmissions, from [7]

The sender will check the channel for activity several times before sending the data. The interval it uses between CCAs is comparable to T_c. It needs to be longer than T_i, but shorter than T_s. This allows the sender to postpone its transmission when it detects that another device is already sending, avoiding collision.

Broadcast messages can not be acknowledged. To be sure that each receiver is able to receive a broadcast message, the sender will send this message for the entire period of the receivers wake up cycle.

2.4.2 X-MAC

Devices follow a strict schedule in which they turn their radio on, and listen for incoming probes. If no probe is received, the radio is turned off for the rest of the cycle. If a probe is received with the correct destination address, the receiver sends an ACK to the sender. After the sender receives this ACK, it can send its data packet [5]. This mechanism is depicted in figure 2.14

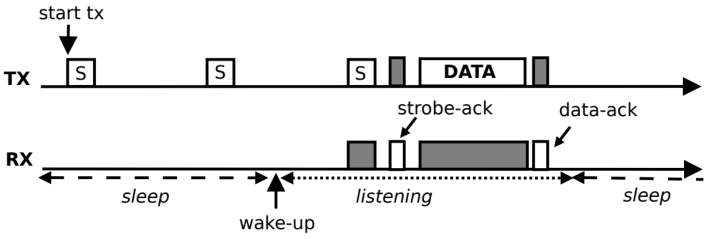


Figure 2.14: X-MAC uses a stream of strobes to advertise a transmission, from [12]

For broadcasting this mechanism is not possible. To be able to broadcast a message, there are two different solutions [5].

In the first implementation, the sender sends probes with a broadcast address for a duration slightly longer than the wake-up period of the receivers. When a receiver receives a probe with the broadcast address, it does not send an ACK. However it keeps its radio on until the sender sends the data package.

In the second implementation, the sender does not send probes but instead sends the data packet multiple times for a duration longer than the wake-up period, to ensure that every receiver is able to receive the packet. This is simpler and has the advantage that receivers don't have to keep their radio on for duration that can be as long as an entire wake-up period like in the first implementation.

X-MAC can also use the phase-lock mechanism. Since the sender receives ACKs from the receivers it is able to keep a schedule of their wake-up times. Because of this mechanism less strobes will have to be sent on average and this will lower the channel utilization and risk of collisions.

2.4.3 LPP

Low Power Probing is a RDC protocol that reverses the roles from sender and receiver compared to low-power-listening RDC protocols like ContikiMAC and X-MAC.

Here it is the receiver that periodically sends probes to let the sender know when the receiver is ready.

To send a packet, the sender has to turn on its radio, listen for for a probe from its intended receiver, and after the probe is received send the data packet. The receiver sends an ACK after the data packet is received.

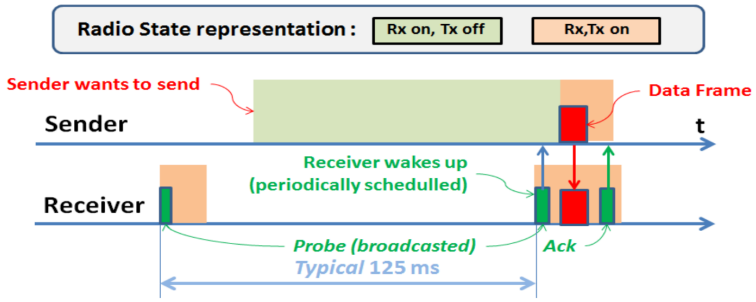


Figure 2.15: LPP unicast transmission, from [23]

The advantage of LPP is that it can be very useful in shared frequency bands to avoid congestion. ContikiMAC and X-MAC will not be able to comply with the restrictions imposed on certain frequency bands with senders that have to repeatedly send requests to send [23].

However LPP also has a glaring weakness. When several devices have to send data packages to the same receiver, their packages will collide after the receiver sends its probe [23].

To broadcast a packet, the sender stays awake for a full wake-up cycle by responding to each probe from the neighbors. However since no ACKs are send for broadcasted messages, this method will suffer the same problem of collisions with no possibility to retransmit with a random delay.

This can be solved by method called "pending broadcast" that can be seen in figure 2.16.

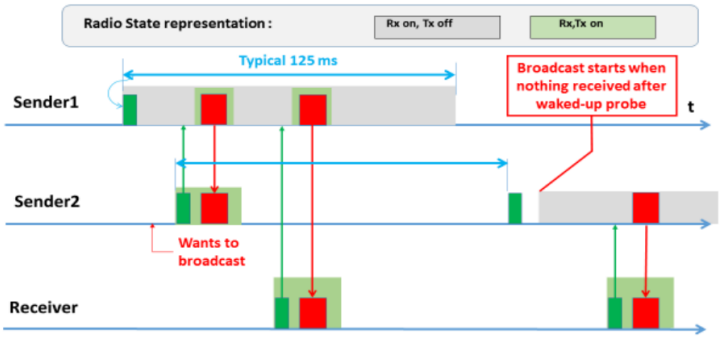


Figure 2.16: Low Power Probing with pending broadcast, from [23]

In this example, sender 2 wants to send a broadcast message. However before doing so, sender 2 sends a probe. Sender 1, who is already broadcasting will receive this probe, and can inform sender 2 that it is already broadcasting. After a while sender 2 can try again. If the probe is not answered it can start broadcasting.

This will increase the latency, but will also increase the chance for broadcast messages to reach their destination [23].

2.4.4 Comparison

For a more complete explanation of the RDC protocols in Contiki-OS and for a comparison and performance analysis, I would like to refer to the PhD thesis written by Marie-Paule

Uwase called "Experimental Comparison of Radio Duty Cycling Protocols for Wireless Sensor Networks" [23].

The thesis concluded that in terms of performance, there were no significant differences between the different RDC protocols that could be brought to light, except for energy consumption, where ContikiMAC performed much better.

However I think it is also important to note that there were numerous protocol malfunctions and malfunctions related to physical properties of supported hardware, like the slightly different clock-frequencies of real-life hardware. Those problems could not be detected with the Cooja simulator used by Contiki.

It also concludes that while simulations are precious and necessary tools, it can't be replace testing with real hardware to make the wireless sensor and actuator networks trustworthy.

2.5 Conclusion

In this chapter, a lot of different concepts were explained.

In the first part an overview of LoRa is given, as well as a more in-depth explanation of LoRa physical layer, LoRaWAN and LoRa 2.4GHz.

To be able to explain the modulation technique LoRa uses as well as why it has such a great resistance against interference, an overview of DSSS is also given.

Although LoRaWAN is not used in this work, it is still important to explain what LoRaWAN is. This to make the seperation between LoRa Physical layer and LoRaWAN the MAC layer. LoRaWAN is also the default MAC layer for LoRa.

The importance of LoRa 2.4 GHz is also shown. This LoRa version that works in the 2.4 GHz ISM band has specific advantages compared to its sub-GHz brother. While the range is shorter, the data-rate can be higher. It also has no duty cycle restrictions and no regional hardware differences are needed.

An overview of Contiki-NG is also given. The radio-driver used in this project is based on Contiki-NG. Therefore an understanding of Contiki-NG is needed.

Lastly an overview of the three RDC protocols from Contiki-OS are given. An understanding of the workings of the RDC protocols is needed, to make an informed choice of which RDC protocol to implement. A comparison is also given, which will make this choice easier.

From the comparison, we learned that while the RDC protocols have a similar performance, contikiMAC has the advantage in terms of power consumption.

In this chapter, an overall overview of LoRa, Contiki-NG and RDC protocols was given. An overview of the hardware and the materials used will be given in the next chapter.

3.1 Introduction

In this chapter the hardware and software used in this thesis will be described.

The Zolertia Firefly was used together with a radio shield containing the SX1280 LoRA 2.4 GHz transceiver. For the communication between the two, SPI (Serial Peripheral Interface) is used. The features of the Zolertia Firefly will be given in section 3.2 and an overview of the radio shield can be found in section 3.3.

In section 3.4 an overview of the transceiver is given. In the subsections an overview of the data buffer is given, as well as the possible operational modes of the transceiver.

At last this section also gives a list with the available commands for the transceiver as well as an overview of the commonly used settings and operations for startup, sending and receiving.

The driver used in this work is based on Contiki-NG [14]. To debug the hardware and the driver, several tools were used. These tools are the HackRF one, a Software Defined Radio (SDR) together with GNU radio, a USB logic analyzer together with Pulseview and a STM32 Nucleo board.

The HackRF is mainly used to detect radio activity. The logic analyzer is used to get an overview of the SPI communication between the Firefly and the radio shield, and the STM32 Nucleo is used to test out the official example of Semtech.

An overview of these tools can be found in section 3.5.

3.2 Zolertia Firefly

The main board used is the Zolertia Firefly. This because the board was available in the lab and it works with Contiki-NG for which a driver was already partly developed. The specific version used is the Zolertia Firefly revision A2. The pin-out of the board can be seen in figure 3.1.

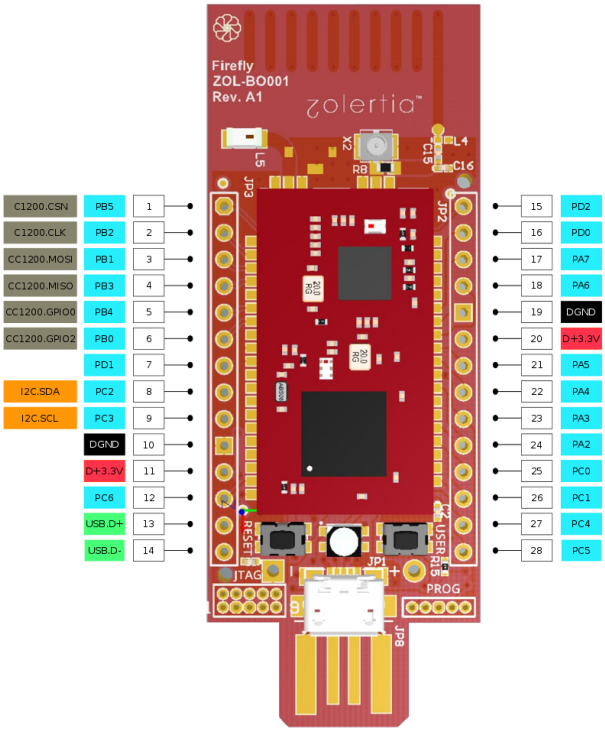


Figure 3.1: Zolertia Firefly pin-out, from [4]

The main features are described on the Contiki-NG documentation page as follows [4]:

- ARM Cortex-M3 with 512KB flash and 32KB RAM (16KB retention), 32MHz.
- ISM 2.4-GHz IEEE 802.15.4 & Zigbee compliant.
- ISM 868-, 915-, 920-, 950-MHz ISM/SRD Band.
- On-board printed PCB sub-1GHz antenna and 2.4Ghz ceramic chip antenna.
- AES-128/256, SHA2 Hardware Encryption Engine.
- ECC-128/256, RSA Hardware Acceleration Engine for Secure Key Exchange.
- Compatible with breadboards and protoboards.
- On-board CP2104/PIC to flash over USB-A connector.
- User and reset buttons.
- RGB LED to allow more than 7 colour combinations.
- Small form factor (68.78 x 25.80mm).

For this thesis, most of the features will not be used, since the main function of the board will be to control and communicate with the shield containing the transceiver. This means that for this thesis, the most important function of the Zolertia Firefly is the SPI communication to the Transceiver and the use of Contiki-NG as its operating system.

3.3 RADIO SHIELD

A radio shield containing an SX1280 transceiver was also needed. This is because the Firefly has no internal LoRa chip.

It was decided to use the LAMBDA80 RF module, which is a high performance, cost effective radio module featuring the SX1280 LoRa Transceiver which can be programmed using SPI.

The LAMBDA80 has been developed together with Semtech to provide a low cost platform application of the sx1280 transceiver. The module has maximum transmit power of +12.5 dBm at 24 mA and works on 3.3 V. The pin-out of the board can be seen in figure 3.2.

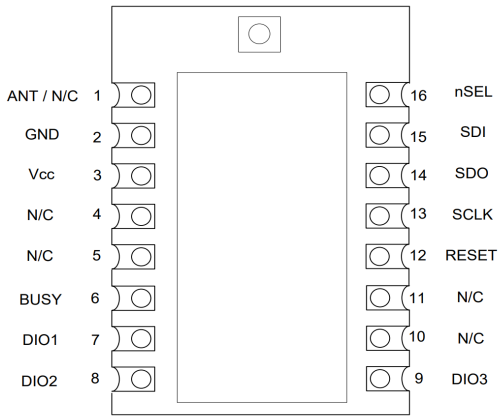


Figure 3.2: LAMBDA80 pin-out, from [21]

Following the datasheet [21] and the application schematic 3.3 a radio-shield was made containing this module. One condition we wanted to fulfill for this shield is that the footprint would not be much larger than the footprint of the Zolertia Firefly. This was only possible because the LAMBDA80 module was just small enough to fit between the headers that fit on the Zolertia Firefly.

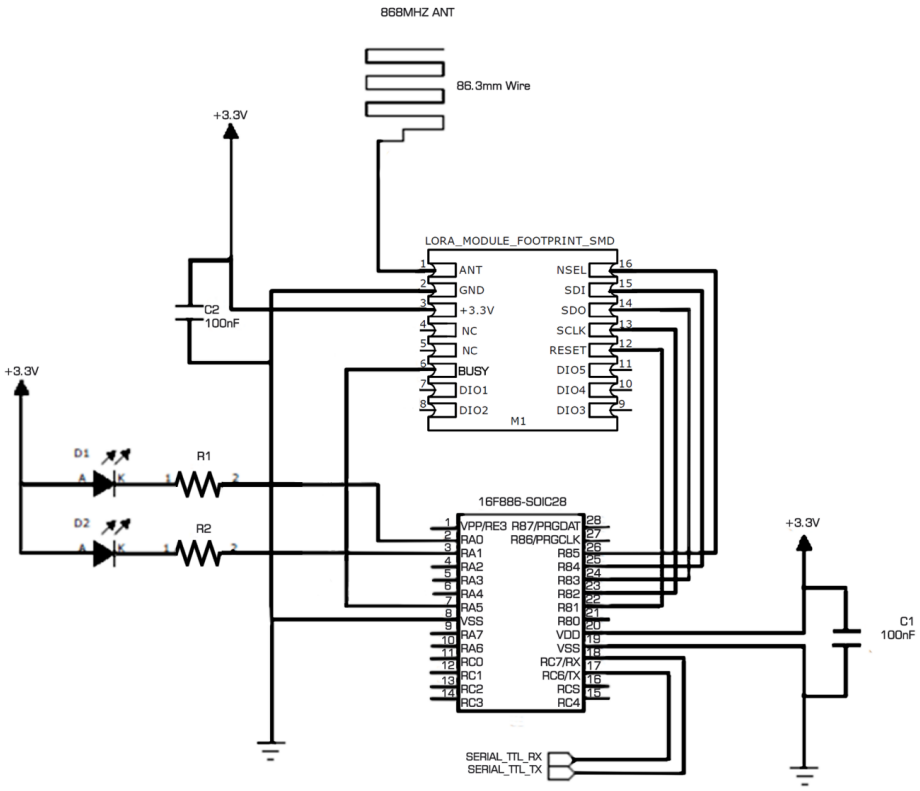


Figure 3.3: Application schematic interfacing a PIC (peripheral interface controller) micro controller, from [21]

The shield made can be seen in figure 3.4. The main difference however was that a reset button was added as well as a capacitor as protection against high frequency changes. The schematic can be seen in the appendices on page 52.

Figure 3.4: Picture from the radio shield containing the Lambda80 module with the SX1280 transceiver

However this shield had a big problem. The MOSI, MISO, CLK and CS pins used for SPI on the Zolertia Firefly to connect with the shield were PB1, PB3, PB2 and PB5 respectively. However these were also already used to communicate with the internal CC1200 chip, which is the radio transceiver for sub-GHz communication integrated on the Zolertia Firefly.

Although the sub-GHz transceiver was not needed for this application, the use of the same SPI pins caused several errors.

My supervisor provided simple breakout boards for the Lambda80. This board can be connected with wires to the Zolertia Firefly. This is not as clean as the previous board, but if there is a wrong connection, it can be changed easily. This board is visible in figure 3.5. I decided to keep working with this breakout module.

Figure 3.5: breakout board containing the lambda80 module

The final set-up is visible in figure 3.6.

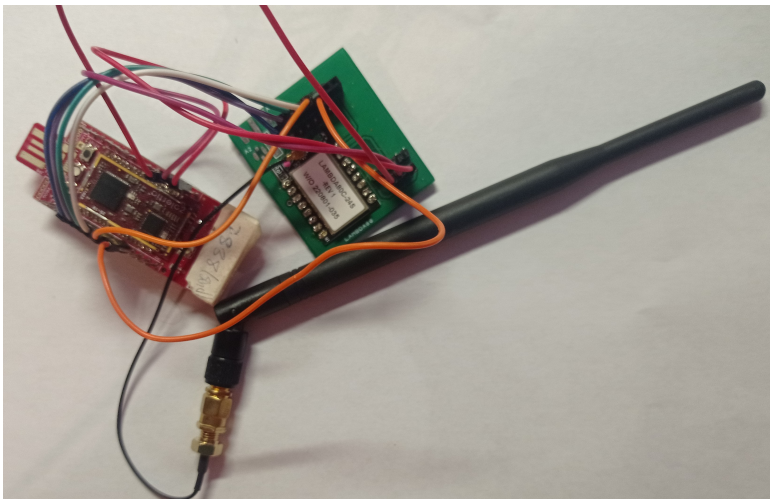


Figure 3.6: Zolertia Firefly connected to a breakout shield containing the LAMBDA80 module with the SX1280 LoRa 2.4 GHz transceiver

3.4 SX1280 TRANSCEIVER

In this section a deeper explanation of the workings of the SX1280 transceiver will be given, as well as an overview of the main features. There is also a more in depth explanation of the inner workings of the transceiver.

The transceiver provides ultra long range communication in the 2.4 GHz band and offers a high resistance against interference. The key features from the datasheet [20] are:

- Long Range 2.4 GHz transceiver
- High sensitivity, down to -132 dBm
- +12.5 dBm, high efficiency PA (Power Amplifier)

- Low energy consumption
- LoRa, FLRC, (G)FSK supported modulations
- Programmable bit rate
- Excellent blocking immunity
- Ranging Engine, Time-of-flight function
- BLE (Bluetooth) PHY (Physical) layer compatibility
- Low system cost

The ranging engine is a method to calculate the time of flight of packets between two devices by an exchange of ranging packets. This feature is not used in this thesis, however it can be very helpful to implement future multi-hop solutions with adaptive SFs.

The data buffer, the operational modes, the available commands and the settings and operations of the transceiver will be closer examined in the following sections.

3.4.1 Data buffer

The transceiver is equipped with a 256 byte RAM (Random Access Memory) data buffer. The RAM is fully customizable by the user and is always accessible except in sleep mode. The principle of operation can be seen in figure 3.7

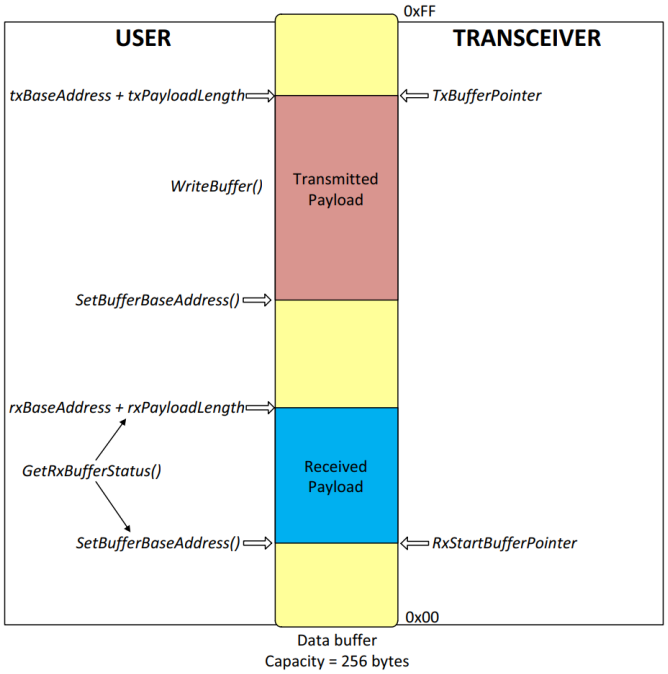


Figure 3.7: Data buffer diagram for the sx1280 transceiver, from [20]

The command SetBufferBaseAddress() is able to set the offset in memory at which a received packet payload will be written. RxDataPointer stores the offset of the last byte written and gets reinitialized to the rxBaseAdress at each new start of reception.

The pointer to the first byte of the last packet received and the packet's length can be accessed with the command GetRxBufferStatus().

For transmission, SetBufferBaseAdress() is able to set the offset for the txBaseAddress. Each time a byte is sent, txBaseAdress is incremented. The transmission stops when the txBuffer-Adress equals the payloadlength parameter set in the function SetPacketParam(). Both buffers are initialized at address 0x00 by default.

The inner workings of the data buffer are not extremely important for the work of this thesis, however it offers a better understanding of the transceiver, which can be useful in case of problems,

3.4.2 Operational modes

The transceiver has six different modes. Those modes are: SLEEP, STDBY_RC, STDBY_XOSC, FS, Tx, and Rx.

On start-up, the transceiver enters a start-up state and sets the BUSY pin to high. It will start calibrations and once those have terminated, the transceiver enters STDBY_RC mode and the BUSY pin goes low. The results of the calibration are stored in its data memory, which means the transceiver does not recalibrate after exiting the sleep mode.

In standby mode, the transceiver can go to Rx or Tx modes. In this mode the transceiver uses the 13 MHz RC oscillator to save energy instead of the 52 MHz of the XOSC (crystal oscillator). In all modes, except the sleep and the standby mode, the crystal oscillator is used. However it is possible to use the crystal oscillator in standby mode with the STDBY_XOSC mode.

In the sleep mode, only start-up and the sleep controller are available by default. This is a mode intended to save power and extend the lifespan of the energy constrained device. This mode can be exited by switching the SPI chip select pin to low.

At the start of the FS (Frequency Synthesis) mode, the BUSY pin is set to high and the signal is synthesized on a new frequency. When this mode is done, the BUSY pin will go back to low.

In Rx mode, the transceiver will look for packets and return to standby mode after receiving one, or after a timeout is reached.

In Tx mode, the data buffer is transmitted, and the transceiver returns to standby after the entire packet is sent or after a timeout is reached.

In figure 3.8 the different states of the transceiver are visible, together with the transition commands.

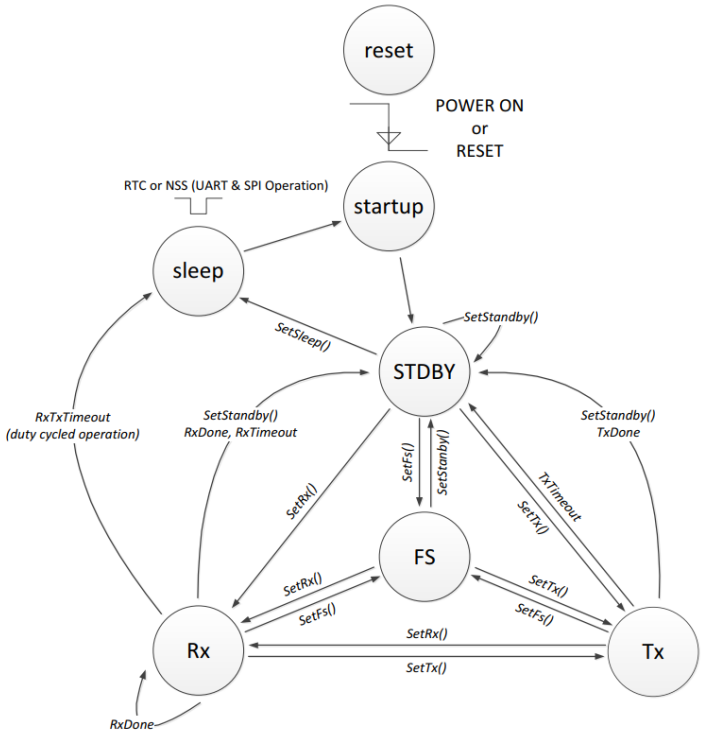


Figure 3.8: Operational modes from the sx1280 transceiver, from [20]

For a more complete explanation of the operation modes I would like to refer to the datasheet of the transceiver [20].

3.4.3 Available commands

For an overview of the available commands I refer to appendix A.2. A more complete explanation of every single command can be found in the datasheet of the transceiver [20].

3.4.4 Settings and operations

To debug the SPI communication between the Firefly and the shield, it is important to know which commands the shield can expect.

Because of that, here is a short overview of the commands expected after startup, for sending a packet and for receiving a packet.

After power up, these are the expected steps:

1. Go to standby mode.
   SetStandby(STDBY_RC)
   This should normally happen automatically after start-up, without this command.
2. Set packet type to LoRa.
   SetPacketType(PACKET_TYPE_LORA)
3. Set the RF frequency.
   SetRfFrequency(rfFrequency)

4. Set the offset for the rxBaseAdress and txBaseAdress.
   SetBufferBaseAddress(txBaseAddress, rxBaseAddress)

5. Set the modulation parameters.
   SetModulationParams(BW, SF, CR)

6. Set the packet format for sending messages.
   SetPacketParams(PreambleLength, HeaderType, PayloadLength, CRC, InvertIQ/chrip invert)

These are the commands needed for sending a message in order:

1. Define the power and ramp time.
   SetTxParam(power,rampTime)

2. Send the payload to the buffer.
   WriteBuffer(offset,*data)

3. Configure the DIOs (Digital Input/Output) and IRQs (Interrupt Requests).
   SetDioIrqParams(irqMask,dio1Mask,dio2Mask,dio3Mask)

4. Set the transceiver in transmitter mode with an optional timeout.
   SetTx(periodBase,periodBaseCount[15:8],periodBaseCount[7:0])
   After the timeout or after the packet has been sent, the transceiver goes automatically back to standby mode.

5. Clear IRQs.
   ClrIrqStatus(irqStatus)

These are the commands needed for receiving a message:

1. Configure the DIOs (Digital Input/Output) and IRQs (Interrupt Requests).
   SetDioIrqParams(irqMask,dio1Mask,dio2Mask,dio3Mask)

2. Set the transceiver in receiver mode.
   SetRx(periodBase,periodBaseCount[15:8],periodBaseCount[7:0])

3. Wait for IRQ RxDone or RxTxTimeout. In case of RxDone, check the SNR (Signal to Noise Ratio).
   GetPacketStatus()

4. Clear IRQs.
   ClrIrqStatus(irqStatus)

5. Get the packet length and offset of the last received packet.
   GetRxBufferStatus()

6. Read the buffer.
   ReadBuffer(offset,payloadLengthRx)

The source of these operations is the datasheet for the SX1280 transceiver [20]. The order of these common operations can be compared with those that the driver sends to the shield in these particular scenarios.

3.5 Debugging tools

3.5.1 HackRF

The HackRF One is an SDR (Software Defined Radio) made by Great Scott Gadgets. It is capable of receiving and transmitting on a frequency range of 1 MHz to 6 GHz, with a maximum output of 15 dBm. It is designed for the testing and development of modern and next generation radio technologies.

Figure 3.9: HackRF ONE

The software used to work with the HackRF One is GNU Radio, a free software toolkit for SDRs.

The HackRF is great for checking for radio-activity in certain frequency bands. With this tool it can quickly be determined if the transceiver is sending messages successfully or not.

3.5.2 Logic analyzer

A logic analyzer is an instrument that is designed to capture signals. A USB logic analyzer can be connected to a PC and relay the information to the logic analyzer software. PulseView was used to fulfill this role. A USB logic analyzer can be seen in figure 3.10.

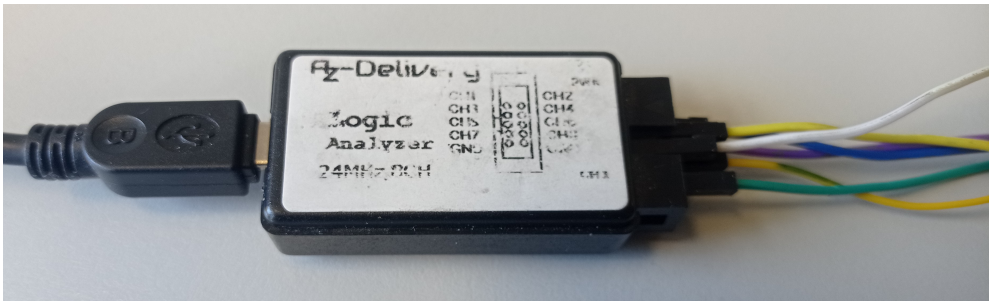


Figure 3.10: USB logic analyzer

Pulseview and the USB logic analyzer can be used to visualize the SPI communication between the Zolertia Firefly and the radio Shield. With this tool, the commands given to the shield can be captured and compared with the commands in the datasheet [20]. Figure 3.11 gives an

example of a captured SPI command to the shield. Only 3 signals are captured, MOSI, MISO and CLK. With those captured signals, PulseView is able to visualise the hexadecimal codes.

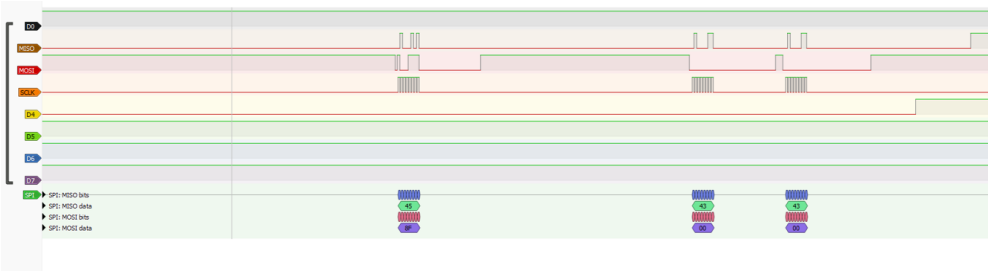


Figure 3.11: SPI communication viewed through PulseView

3.5.3 STM32 Nucleo board

The STM32 Nucleo boards provide a way to build prototypes for any STM32 micro controller line. All the external hardware necessary for the complete feature set of the micro controller is included on those boards. This means that it is an excellent tool to build a prototype for an application using this micro controller.

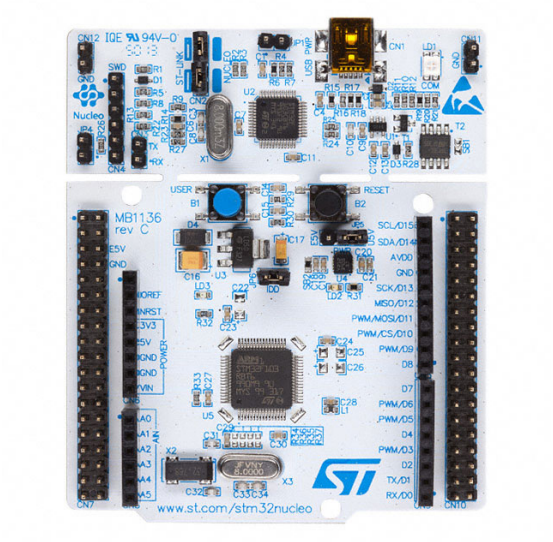


Figure 3.12: STM32 Nucleo board

However the main reason that this board is used, is that Semtech made an official example for the sx1280 transceiver, called "SX1280PingPong", using Mbed 2 as OS. The Nucleo board using an STM32 micro controller is supported by MBED 2. This means that it is a great tool for testing the official example.

3.6 Conclusion

This chapter gives a more in-depth overview of the hardware used in this work. An overview of the Zolertia Firefly is given in section 3.2, as well as an overview of the radio shield in section 3.3. The Firefly is used together with the shield, as the main hardware for this project.

Some of the main functions, as well as the most important operations of the sx1280 transceiver are also explained. Especially the operations are important for the debugging of the SPI commands of the transceiver. Information about the transceiver can be found in section 3.4.

The order of the operations and the common settings given in the datasheet can be compared with the commands given to the transceiver by the driver for debug purposes.

Lastly, an overview is given of the main debugging tools used in this project in section 3.5. The HackRf is used to detect activity from the transceiver, the USB Logic Analyzer is used to debug the SPI commands, and the STM32 Nucleo is used to test the official example.

4.1 Introduction

To be able to use LoRa 2.4 GHz with the Zolertia Firefly boards running Contiki-NG, the SX128x transceiver driver [14] was used. This driver is a preliminary implementation, and can be further enhanced and debugged.

In this chapter, a short overview of the driver is given, as well as the problems I experienced and the enhancements I made when testing the driver.

4.2 Driver directories

The driver consists of a directory for Contiki-NG, the OS. A directory for the examples using this driver and a directory called "src" for the driver itself.

In the directory "src" the following files can be found:

- lora24.h: here the default LoRa physical paramaters are defined (E.g. SF, BW, preamble length, ...)
- sx128x.c: implementation of the main radio.h functions. See section 2.3.4. These functions make use of the functions from sx128x_getset to configure the transceiver.
- sx128.h: interface for the radio driver. A lot of the settings of the transceiver are changed here (E.g. the pins used to connect the transceiver to the Firefly, timeout settings, Tx power settings, initialisation BW, SF, etc...). The declaration of the functions for the driver, as well as for the get and set operations can be found here too.
- sx128x_getset.c: all the get and set functions can be found here. These are the functions that configure the spi commands to be sent to the transceiver. The main radio.h functions, call these functions to configure the transceiver as wanted.
- sx128x_internal.c: here the functions to communicate directly with the transceiver are defined. sx128x_cmd_burst() is defined here, which is the function that sends the configured spi commands to the transceiver. This function is almost always called in the get-set functions with the matching commands. The other functions are meant to read or write to the internal memory of the transceiver.
- sx128_xinternal.h: here are the functions from sx128x_internal.c declared.
- sx128x_registers.h: in this file, the hexadecimal commands for the transceiver can be found, as well as the hexadecimal codes for the intended settings (e.g. BW, CR, SF, CAD symbols, IQ, IRQs, ...).

There are only 2 examples given in the example file. An example called "SX1280" and an example called "TSCH".

The TSCH example will not be explained, since it is outside the scope of this work. The example "sx1280" uses shell commands to perform certain actions. The commands are:

- send: This function sends a basic hello-world message by default. It is also possible to send a custom message by typing "send message", where "message" is the actual message you want to send.
- receive: This function turns the radio on and checks every 0.5 s if there is a valid message in the buffer.
- toa: This function calculates the ToA for a given payload.

4.3 Radio driver internal state machine

When looking at the driver in the subdirectory "sx128x/src/sx128x.h", there are 4 possible states of the radio driver that can be found, as well as 8 operational modes.

The states are called:

- SX128X_RF_IDLE
- SX128X_RF_RX_RUNNING
- SX128X_RF_TX_RUNNING
- SX128X_RF_CAD

The operational modes are called:

- sx128x_mode_sleep
- sx128x_mode_standby
- sx128x_mode_synthesizer_tx
- sx128x_mode_transmitter
- sx128x_mode_synthesizer_rx
- sx128x_mode_receiver
- sx128x_mode_receiver_single
- sx128x_mode_cad

There is an overlap between the modes and the states.The function `sx128x_set_op_mode()`, that sets the operational modes, is not able to set the mode to a synthesizer mode. At least, it is able to, but it won't do anything. There is also no difference between the mode "receiver" and receiver_single. Both modes will call the same functions that will send the exact same command to the transceiver.

If the other files are searched for the synthesizer modes, no results show up. This means that those modes are not used. So instead of 8 operational modes, there are actually only 5. These are the same modes as the states, with the addition of a sleep mode.

The states are only an abstract layer and bring no addition to the workings of the driver itself. The function sx128x_set_state() only changes the name of the state. It does not send commands to the transceiver, or make calls to other functions. The radio states are also not

used anywhere in the driver to get the operational mode of the transceiver at the current moment. For this, the operational modes are used.

The operational mode of the transceiver is used when an interrupt occurs. When the operational mode of the transceiver is determined the meaning of the interrupt can be more easily found. The function sx128x_set_op_mode() makes calls to other functions when a new operational mode is set. For example, when the mode is set to "receiver_single", sx128x_cmd_set_rx() is called.

So while the operational modes and the states are very similar, they are not quite the same.

It is also possible to get the status of the transceiver via the command getStatus(). However this function is not really used in the driver. This command is executed once in the initialization function of the radio driver, but the result is not used nor saved.

The modes and their transition can be seen in figure 4.1.

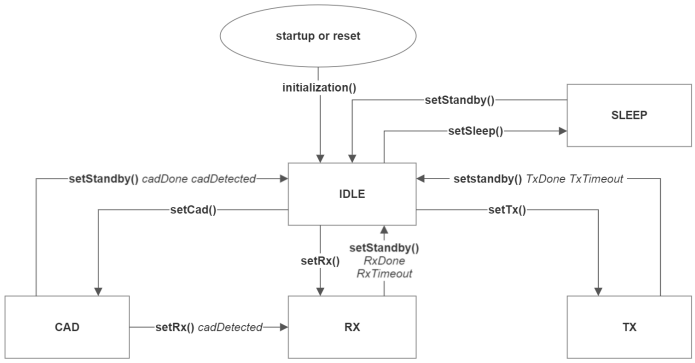


Figure 4.1: radio driver operational modes

4.4 Interrupt request

Interrupt Requests (IRQs) is the method used to go from the operational modes CAD, Tx and Rx, back to the IDLE state. The IRQ sources can be set when changing from state. For example, when the Rx mode is enabled, the IRQs RxDone and RxTimeout are set. Depending on which IRQ is asserted, it is possible to know if a message was correctly receiver (RxDone), or not RxTimeout. The IRQS used in the driver can be seen in figure 3.8.

There are 2 main methods to know which IRQ is asserted. The first method, this is the method that was used in the driver by default, uses a while-loop to periodically request the IRQs status. The second method uses an interrupt pin. Such an interrupt pin can be used for more than one different IRQ. When the pin goes high, an interrupt is detected and the IRQ status is requested only once from the receiver.

Personally I prefer method 2 since it will cause a lot less overhead compared to method one. Especially when you want communication to happen fast. For example, in some RDC protocols like contikiMAC, an ACK is send after receiving a message. This ACK has some timing constraints and has to be send as fast as possible and be received as fast as possible by the original sender. See the timing constraints in chapter 2 in section 2.4.1.

The biggest problem with the while-loop is that the watch-dog timer is running without being refreshed. This means that a restart will be triggered if the loop is not left within a certain time frame. This can be circumvented by adding a delay in the loop and by adding "watch-dog_periodic" , which will refresh the timer. This delay can be for example 1 ms. This means

that every 1 ms the status will be requested. For long messages this can result in hundreds of status requests. A lot of unnecessary overhead.

However the delay can be made longer to solve this issue. But this has its own problem. If the delay is changed to 10 ms for example, then this means that the IRQ status RxDone can be received 10 ms later than that the transceiver was actually done with receiving the message. Now that means that the receiver will start sending its ACK of the received packet 10ms too late. This is not really desirable.

4.5 Driver testing

In this section, I will explain the steps I took to test the driver and the changes I made while getting it to work.

4.5.1 Installing and setting-up

To test the driver, a radio shield was developed containing the sx1280 transceiver. More about this shield can be found in chapter 3, section 3.3.

After connecting the shield to the Firefly, it was time to test the driver. To work with the driver, Virtualbox with an Ubuntu VM was used. After the driver was installed, the example "sx1280" was set up, by adding the correct board and target to the makefile.

The lines added are:

- TARGET=zoul
- BOARD=firefly

How the example works is better explained in section 4.2. Another change that had to be made was changing the default pin connections of the driver to these I used to connect the shield.

This could be done in the "sx1280.h" file of the driver.

After setting this all up, the driver was ready to be tested. This could be done with the shell command "send" and "receive". However after trying this out with 2 boards, one sender and one receiver, no messages were received.

4.5.2 Debugging

To be able to see if messages were actually transmitted at the sender, the HackRF One was used together with GNU radio. But after testing this, no radio activity was detected.

It was time to measure the SPI commands send to the transceiver and analyze them. Since no radio activity was detected, we decided to measure the SPI commands with the USB logic analyzer from the startup phase and the send phase and cross-analyze them with the data-sheet.

This gave the following commands for the startup phase:

- 0xc0 0x00: getStatus
- 0x96 0x01: setRegulatorMode (DC-DC)
- 0x80 0x00: setStanby
- 0x8a 0x01: setPacketType (LoRa)
- 0x8b 0x70 0x34 0x01: setModulationParams (SF 7, BW 203 kHz, CR 4/5)

- 0x18 0x09 0x25 0x37: writeRegister
- 0x8c 0x08 0x00 0x00 0x20 0x40 0x00 0x00: setPacketParams (preambleLength 8, headertype explicit, payloadlength 0, CRC enable, IQ, NU, NU)
- 0x86 0xb8 0x9d 0x89: setRfFrequency (set to 2.4 GHz)
- 0x8f 0x00 0x00: setBufferBaseAdress
- 0x8e 0x1e 0xe0: setTxParams (power, rampTime)
- 0x8d 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00: setDioIrqParams

These commands for the send operation were found:

- 0x8c 0x08 0x00 0x0a 0x20 0x40 0x00 0x00: setPacketParams (preambleLength 8, headertype explicit, payloadlength 0, CRC enable, IQ, NU, NU)
- 0x8f 0x00 0x00: setBufferBaseAdress
- 0x1a 0x00 0x68 0x65 0x6c 0x6c 0x6f 0x77 0x6f 0x72 0x6f 0x72 0x6c 0x64: writebuffer
- 0x97 0xff 0xff: clearIrqStatus (all flags)
- 0x8d 0x40 0x01 0x40 0x01 0x00 0x00 0x00 0x00 0x00: setDioIrqParams
- 0x83 0x00 0x00 0x00: setTx
- 0x15 0x00 0x00 0x00: getIrqStatus (this one repeats a lot)

In chapter 3, section 3.4.4, the expected operations for startup and send are shown. When compared with these results, some differences were found. In the datasheet, setTxParams() is called for every send operation again, while it is not called in the startup. GetStatus, Writeregister, setRegulatorMode and setDioIraParams were also not called in the datasheet, however these should not make a difference. The startup commands in the datasheet are only a guideline after all.

No immediate faults were detected from the SPI commands, and the only changes made to the driver were placing the "setTxParams()" function inside the send operation and changing the getIrqStatus from the send operation that kept repeating, to an interrupt. However while this was more like the guideline from the datasheet, and there were no getIrqStatus requests being send all the time, this did not solve the problem.

The next step was to test an official example of Semtech called "sx1280PingPong". However this example was written with Mbed 2 OS, which does not support the firefly boards. So the STM32 Nucleo boards from the lab were used. To be able to upload the example to the Nucleo board, Keil MDK (Microcontroller Development Kit) was used.

After the correct connections between the Nucleo and the shield were made, the example could be tested and checked for radio activity.

Radio activity was detected, and the next step was to measure the SPI commands with the USB Logic Analyzer. However it was soon found out that the system malfunctioned with the Logic Analyzer connected. Testing each cable connected to the analyzer separately, it was found that the MOSI connection between the radio shield and logic analyzer made the transceiver malfunction.

At that moment, the issue was solved by placing a diode before the logic analyzer together with a pull-down resistor. This stopped the issue and the transceiver started working again.

Later I found that when the Logic Analyzer causes issues on the signal that it is recording that this is referred to as "loading". There are several issues that can cause this, and several possible solutions. However since the data was already measured and the solution used worked, the issue wasn't studied any further.

More information on loading can be found here [22].

These are the results for the startup:

- 0xc0 0x40: getStatus
- 0x96 0x01: setRegulatorMode (DC-DC)
- 0x80 0x00: setStanby
- 0x8a 0x01: setPacketType (LoRa)
- 0x8b 0x70 0x26 0x01: setModulationParams (SF 7, BW 406 kHz, CR 4/5)
- 0x8c 0x08 0x00 0x0f 0x20 0x40 0x00 0x00: setPacketParams (preambleLength 8, header-type explicit, payloadlength 0, CRC enable, IQ inverted, NU, NU)
- 0x86 0xb8 0x9d 0x89: setRfFrequency (set to 2.4 GHz)
- 0x8f 0x00 0x00: setBufferBaseAdress
- 0x8e 0x1f 0xe0: setTxParams (power, rampTime)
- 0x8d 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00: setDioIrqParams
- 0x97 0xff 0xff: clearIrqStatus (all)

Some settings were different. The power used in setTxParams is a value higher, the BW is 406 kHz instead of 203 kHz, the preamble length is 15 instead of 8, IQ inverted is used and the default payload length is set to to 15 instead of 0.

The results for the send operation were:

- 0x15 0x00 0x00 0x00: getIrqStatus
- 0x97 0xff 0xff: clearIrqStatus (all flags)
- 0x8d 0x40 0x01 0x40 0x01 0x00 0x00 0x00 0x00 0x00: setDioIrqParams
- 0x97 0xff 0xff: clearIrqStatus (all flags)
- 0x1a 0x00 0x50 0x49 0x4e 0x47 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00: writebuffer
- 0x83 0x02 0x00 0x64: setTx (timeout added)
- 0x15 0x00 0x00 0x00: getIrqStatus
- 0x97 0xff 0xff: clearIrqStatus (all flags)

The main difference between this official example and the driver is that here, the send operation does not set the packet parameters and the Tx parameters. All these parameters are only set once in the startup.

This means that setTxParams can be put back into the powerup operations instead of the send operations.

Another difference is that this example makes use of a timeout for Tx. This causes the transceiver to leave the Tx state after a certain duration, even if it was not successful.

The settings used in the official example were tried out for the driver, however no activity was detected.

4.5.3 Solution

After more testing and changes, it was finally discovered that the HackRF sometimes measured radio activity, however not always.

Finally one of the main issues was found. The SPI pins used to connect to the shield were already used by the internal CC1200 chip of the firefly used for sub-GHz communication. This issue could be solved by changing the Chip Select pin to another pin. However this also meant that the shield didn't fit on the Firefly anymore.

Luckily my supervisor was able to provide breakout shields for the lambda80s, which could be used for further testing.

However, after changing the pins, the board still malfunctioned. Now no radio activity was detected at all. This led me to believe there was a hardware issue.

After testing the shield's connections with a multi-meter, no problems were found. However I still decided to test with another shield. This time radio activity was detected.

The only possible explanation why the first shield didn't work, i can think of, is that the Lambda80 module on the first shield was defect.

After re-trying the example project, still no messages were received. This was likely a software issue, since messages were actually being sent but not received.

However after making changes and testing the receiver side multiple times, I decided that to solve this issue, better examples were needed.

The example file is annoying for testing in my opinion. One device has to manually send a message, and the other has to be put manually in receive mode.

For working with this driver I made use of a VM, and the Firefly boards have a lot of trouble connecting via USB to the VM. This led to a lot of plugging-in and plugging-out of the devices to get them to work.

On top of that the commands had to be manually given, which meant that 2 devices had to be connected all the time. When a device randomly disconnected from the VM, led this to a lot of wasted time, which was not really efficient for testing.

For this reason, 2 new examples were written. The fist example is called "sx1280_send" and the second example is called "sx1280_recv".

In the examples sx1280_send, a "hello-world" message is periodically sent. A counter is added to the message. The counter counts up for each message sent. This makes it easier to see if messages get lost.

In the example sx1280_recv, the transceivers radio is turned on. After that the transceiver waits for the RX_DONE or RX_timeout interrupt. A timeout can be added to the receiver with the setRx() command.

If the RX_DONE interrupt is activated, the received message is read out. In the case of a timeout, or after the message is read out, the radio is turned off for a short while and is then turned back on and the cycle repeats.

After setting up, two boards with these examples, messages were finally being received.

Because of all this debugging and getting everything to work, I learned a lot about Contiki-NG and the Driver.

I believe the documentation for Contiki-NG is sometimes lacking and can be further improved upon. Although a lot of the documentation can be found on the official documentation page of Contiki-NG, there were still some things I had trouble finding. An example of this is the construction of radio drivers for contiki-NG which I found inside the "radio.h" file and not on the official documentation page.

I also believe the LoRA drivers documentation could be further improved upon. The main documentation of the driver are comments inside the source code. I added to this documentation by adding extra comments where I thought they could be helpful.

4.6 Changes and settings

While testing the driver, multiple different pin-outs were tried for debugging purposes.

The pins last used were:

PIN	**id**
SCK	B2
MISO	B3
MOSI	B1
CS	A3
RESET	A2
DIO1	A5
BUSY	A4

Table 4.1: Pin connection between the Zolertia Firefly board and the radio shield.

Also multiple different radio-settings were used while testing. The settings last used were:

setting	**value**
SF	7
BW	403 kHz
Preamble length	8
CR	4/5
CRC	true

Table 4.2: LoRa settings used in the radio driver.

The first change made to the driver was the enabling of interrupts. The interrupts I use are RX_DONE, CAD_DONE, CAD_DETECTED, TX_DONE and TX_TIMEOUT and RX_TIMEOUT. To those interrupts variables were added, whose values get changed when an interrupt happens. For example, when the interrupt RX_DONE is activated, the interrupt sets the variable RX_DONE_FLAG to 1. This variable can be used in projects to keep track of which interrupt happened.

For example, previously when the interrupt CAD_DETECTED was activated, the driver put the radio automatically in receiving mode. However for projects where listen before talk is used, this will not be useful, since in this case, setStandby() should be called and a random time later, the CAD should be activated again.

Now CAD_DETECTED_FLAG is set to 1, and the user can check those variables and decide what happens next.

An issue was also found when multiple interrupts happen at the same time. The interrupts are defined as hexadecimal numbers, and when the IrqStatus() is requested from the transceiver, such a number can be received and compared with the possible interrupts.

CAD_DONE has a decimal code of 4096 and the code for CAD_DETECTED is 8192. Each interrupt correspond with a binary word, existing of a '1' followed by zeroes. The position of the '1' determines the interrupt.

If both interrupts happen at the same time, both their interrupt bits are set to '1', and the number received is 12288. For the interrupts used, these are the only 2 interrupts that should possibly happen at the same time. And the issue could be resolved by using 12288 or CAD_DONE + CAD_DETECTED instead of only CAD_DETECTED. This solution works because CAD_DONE is always triggered when CAD_DETECTED occurs.

The setRX() call that puts the transceiver in receiving mode was also added to the sx128x_on() function. Normally without the interrupts, messages are being received by using the functions receiving_packet() and pending_packet(). Where receiving_packet() will check if there is a packet ready to be received, and pending_packet() if this packet can be read out.

However when working with interrupts, the role of these functions can be handled by them. The only thing that needs to happen is for the radio to be put in receiving mode. When a packet is then ready to be read, the RX_DONE flag is triggered, and we now that the packet can be read.

The last important change I made, is to let the "clear_channel_assessment()" function run a CAD. Previously, the function returned always CCA_CLEAR, but in actuality did nothing. It was written that this was done so that the driver will always transmit. However the function is not really used in the rest of the driver, so I see no reason for this. Maybe this function is used in the TSCH example.

I also added an extra example called "sx1280_cad". Which is an example that first does a CAD and when CAD_DETECTED triggers, activates the radio to receive a message. This example works together with "sx1280_send" for the transmitter.

4.7 Conclusion

This chapter gives a short overview of the driver, the problems I experienced as well as the steps I took to get it fully functional with the new hardware. It also gives a short overview of the main changes made to the driver.

It explains the difficulties I encountered for making the driver work with the available hardware. These difficulties arose from problems with the driver itself as well as from problems with the new hardware.

Because of the problems I encountered, I learned a lot about Contiki-NG and the driver itself.

The driver is mainly documented with comments in the source code, and I believe that this can be further improved. I started this by adding extra clarifications in the source code where I thought that they could be beneficial.

There were also three test programs added to the driver. These test programs consist of an example for a device that periodically transmits messages, an example that continuously receives messages and an example that uses CAD to detect messages before receiving them.

5.1 Introduction

One of the goals of this thesis was to implement a preliminary version of an RDC protocol for the LoRa 2.4 GHz driver for Contiki-NG. 3 RDCs were explained in chapter 2. Those RDC protocols were contikiMAC, X-MAC and LPP.

From [23], we learned that the RDC protocols have a similar performance. The main difference is that contikiMAC has a much better energy efficiency compared to the others.

Because of this reason contikiMAC is the best option. However contikiMAC makes use of CCAs. This is a problem because LoRa can be demodulated under the noise floor.

One of the alternatives for the CCAs we were thinking about was preamble detection. However this has the problem that the preamble is only sent at the start of a packet, which destroys the purpose of a short CCA to find a packet.

One of the solutions for this, is to keep sending the preamble instead of the packet.

However now we are closer to X-MAC. The packet being sent isn't a packet anymore but more of a probe. The CCA isn't a CCA anymore, but more of a probe detection.

For this reason I started working on a version based on X-MAC instead of contikiMAC. However I later found out that the CAD function of the sx1280 transceiver works different than I expected.

Since spread spectrum modulation techniques make it challenging to determine if a channel is already in use, the CAD of LoRa actually detects the presence of LoRa signals. It is used to detect an incoming LoRa frame by performing a correlation on received samples. Its primary function is low-power preamble detection, however it is also able to reliably detect payload chirps of ongoing transmissions [19].

This made it possible to develop a contikiMAC version.

5.2 contikiMAC implementation

For the preliminary implementation of contikiMAC for LoRa, I implemented the RDC protocol in a project instead of in the correct MAC layer of Contiki-NG.

The reason for this was mainly for ease of development, and to have a working protocol as fast as possible.

Contiki-OS had a separate RDC layer, where RDC protocols could be implemented. However Contiki-NG deleted this layer, and added the RDC layer and MAC layer back together. This made it more difficult to integrate a RDC protocol, since there are not really any examples available in Contiki-NG about how to write an RDC protocol on the MAC layer. The closest example would be to look at the outdated Contiki-OS for examples of separate RDC layers and how they fit in the operating system. However I could find very little documentation about

this process.

To make a project that simulates an RDC protocol would be much simpler. However I wanted to make sure that the protocol could be transported easily to other projects.

To achieve this I decided to add two process_threads to the project and both processes will auto-start on the start-up of the project. One process, will simulate the RDC protocol and the other process will simulate the user interface. The user interface will mainly add messages to be sent, while the RDC process will handle when these messages will be sent and the reception of the messages.

The RDC process works with 2 callbacks that get activated periodically. One Callback handles the reception of messages with 2 consecutive CADs, while the other callback handles the transmission.

The callback for reception is activated every 100 ms. Two consecutive CADs are done and if the CAD_DETECTED interrupt is triggered, the radio is turned on for reception. If a packet is detected by the RX_DONE interrupt, an ACK is immediately send to the sender.

If no CAD was detected, the transceiver waits for a new cycle. If the CAD_DETECTED interrupt was triggered, but no packet was received, the transceiver will turn the radio off after 500ms.

To periodically send a message, the message buffer is checked. If there is a message in the buffer, the transceiver starts transmitting. After each message send, the radio is turned on for a short duration to receive an incoming ACK message. If an ACK is received. The Radio is turned off and the message is deleted from the buffer.

If no ACK is received, the transceiver starts transmitting the message again. It repeats this process until an ACK is received, or if he failed to receive an ACK more than 10 times. If no ACK was received, the message is not deleted from the buffer and will be sent again in a next cycle.

The user process is used to add payloads to the buffer that can be sent with the RDC protocol. Messages will be received automatically.

To make this protocol, the following functions were used:

- add_payloadBuffer(): adds a packet to the buffer
- clear_first_item_from_payloadBuffer(): clears the first packet from the buffer. This functions is called when an ACK of successful transmission is received.
- send(): this function sends the payload multiple times until an ACK is received, or until the function loops more than 10 times.
- cad_check(): this function carries out a CAD and returns 1 if a CAD was detected
- receive(): this function activates 2 consecutive CADs and activates the radio if a CAD was detected
- receive_callback(): this is a callback that is triggered periodically in the main process by a timer. this callback function starts the receive() function if we are not already sending.
- send_callback(): this is the same as for the receive callback, but is used to activate the send() function.

The timing constraints can be found in chapter 2 in section 2.4.1.

The timings used in this project are:

- Ta + Td = 30 ms (The time between the end of the payload transmission and the complete reception of the ACK)
- Ti = 33 ms (The time interval between two consecutive transmissions)
- Tc = 36 ms (The time between two CADs)

For the first test of the protocol I used Ta + Td = 20 ms. However this proved to be too little, since an ACK was never received. Since this value has to be the lowest value following the timing constraints, all the other values have to be higher.

Ts, the time needed to send the payload, needs to be longer than Tc + 2Tr, which is the time between two CADs and the time needed to carry out those 2 CADs.

I logged the time before a CAD and the time after. This showed me that a CAD with these settings took around 7ms. This means that Ts needs to be at least 50ms.

A quick calculation with the LoRa symbol time $2^{SF}/BW = 2^7/(400kHz) = 0.32ms$. The payload needs to be at least 156.25 symbols long for each LoRa symbol with SF representing 7 bits. So this means that the payload needs to be at least 1093.75 bits, or around 137 chars.

However in reality this number can be less. This number doesn't take into account the preamble symbols, nor the error correction bits. I also added a time stamp to the messages that also add some symbols.

It is important to note that this system is not designed to send short messages.

In figure 5.1, the main overview of the RDC protocol can be seen.

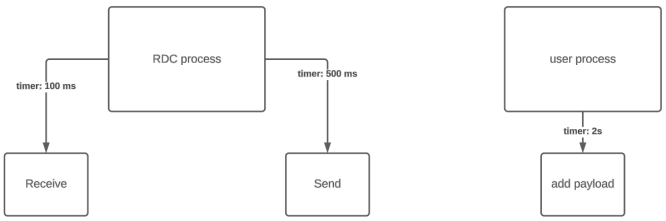


Figure 5.1: main overview of the implemented RDC protocol

In figure 5.2, an overview of the receive operation is given.

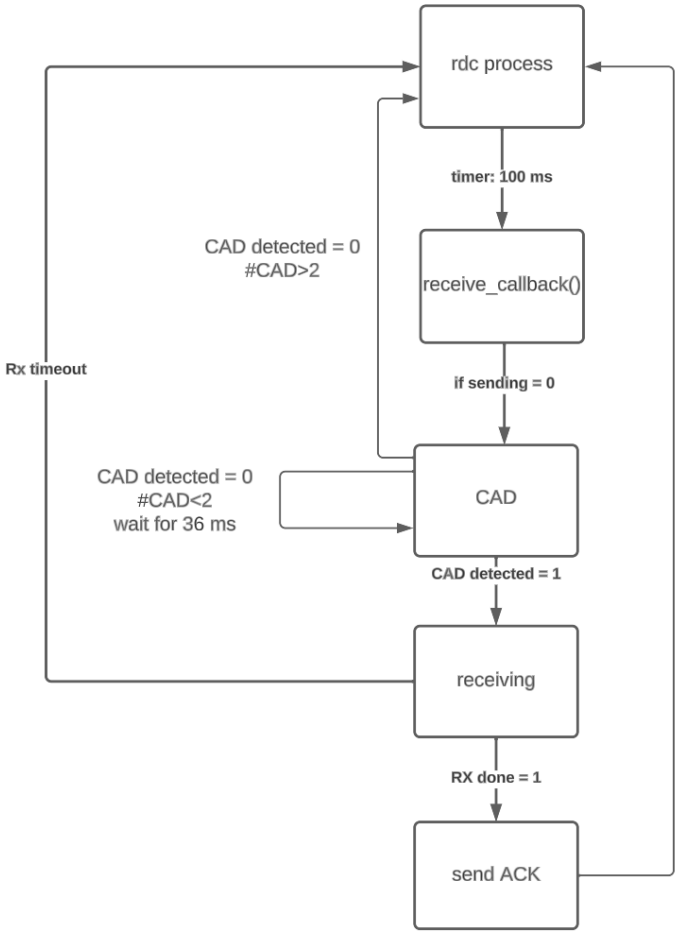


Figure 5.2: receive overview of the implemented RDC protocol

In figure 5.3, an overview of the send operation is given.

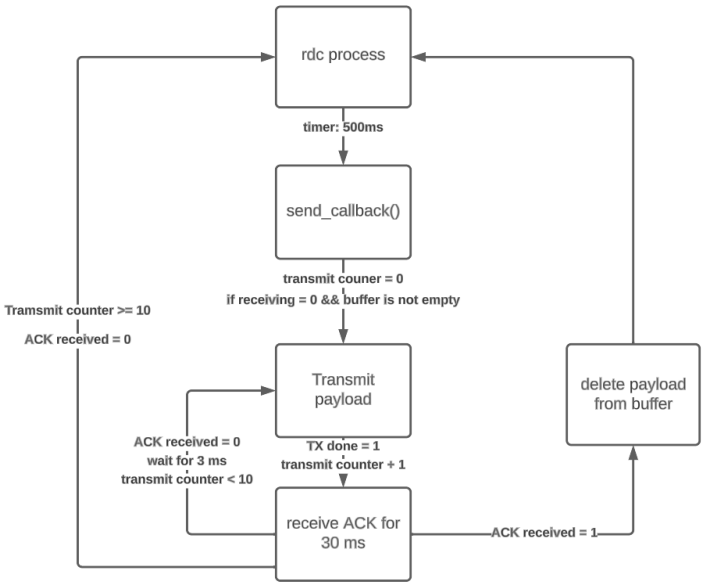


Figure 5.3: send overview of the implemented RDC protocol

5.3 Discussion

The workings of the CAD operation of LoRa really opened the door to new possibilities. I already knew that the CAD operation was able to do preamble detection, however no source I read, mentioned that it was able to recognize payload packets.

After finding this official Semtech introduction to CAD for LoRa [19], I searched the datasheet of the sx1280 transceiver again for a detailed CAD explanation for this specific transceiver. However I did not find any explicit mention that it was able to do preamble detection, nor packet detection.

In my opinion the workings of the CAD operation should be better explained in the datasheet.

But because of this CAD operation I was able to implement an adaptation of contikiMAC for LoRa 2.4 GHz, that in a relatively simple way can be implemented in other projects.

To allow communication between devices, each transceiver is only in CAD mode for approximately 7 ms for each 100 ms. This means that the transceiver is in the energy efficient CAD mode approximately 10% of the time. The transceiver only uses more power-hungry modes, like the Tx and the Rx mode when packets are being detected, or when transmitting.

This is a big improvement compared to simply keeping the radio always on.

However it would still be possible to further optimize this RDC protocol. The timings could be optimized, and the cycles could be made longer.

It is also possible to increase or decrease the number of symbols LoRa CAD tries to recognize for a CAD detected interrupt. I have used 4 symbols, but it is possible to go down to 1 symbol. However the lower the number of symbols to be detected, the higher the error rate.

The send operation period should also be randomized to limit the number of collisions. Alternatively, listen before talk could also be implemented before the start of a transmission.

Another improvement that could be made is the automatic change of the timings for other SFs or BWs. As it stands now, the example might not work if another SF or BW is used because

of the timing constraints.

5.4 CONCLUSION

The implemented RDC protocol, showed that CAD could be used for the implementation of an alternative version of contikiMAC for LoRa.

As the RDC protocol is implemented as a project, it can simply be ported to other projects. Although the protocol works well, a lot of optimizations can be made. The timings could further be optimized, as well as the number of LoRa symbols to be detected.

The protocol can also be further tested with different SFs and bandwidths and with more devices. If this would perform well, a full RDC layer protocol for LoRa could be made with this preliminary version as the basis.

6.1 Conclusions

The objectives of the thesis were the following:

- study LoRa and LoRa 2.4 GHz
- get more familiar with Contiki-NG and the driver [14] for the sx128x transceivers
- make a hardware shield containing the sx1280 transceiver for the Zolertia firefly
- test and evaluate the driver and enhance and debug where needed
- choose/design, implement, test and evaluate a preliminary version of an RDC protocol for LoRa 2.4 GHz

Most of the goals have been accomplished. The radio driver was successfully tested and is fully functional. Also, a preliminary version of an RDC protocol based on contikiMAC was implemented.

The process of getting familiar with Contiki-NG and the radio driver for the sx1280 transceiver was challenging and required a lot of knowledge about protocols, radio drivers and the transceiver.

The documentation for Contiki-NG and the driver was sometimes lacking and I started adding comments in the source code where I thought that extra clarifications could be beneficial.

A radio shield was also designed and manufactured, however there was an error in the design. The Chip Select pin for the SPI communication between the Firefly and the shield was also used for the internal CC1200 chip. Therefore an alternative breakout shield was provided and used.

A full analysis of the driver was made, as well as some improvements. Three test programs were added that show some of the functionalities of the driver. In the first, a device periodically transmits LoRa messages, in the second, a device continuously receives LoRa messages and the last one makes use of CAD to detect messages before receiving them.

Lastly an RDC protocol based on ContikiMAC was developed. ContikiMAC was chosen because of its energy efficiency compared to the other RDC protocols. The implemented protocol can easily be ported to other projects and shows a working proof of concept.

Despite these successful implementation of the protocol, it still requires further in-depth testing and optimization.

As an overall conclusion, most of the goals have been completed, however there is still further work to be done. The possible enhancements that can be made will be explained in the next section.

6.2 Future work

Despite the fact that the driver works well and fulfills its main task, I believe that there are still improvements possible. I also think that the documentation, although already available thanks to my work can still be extended . Indeed, I added extra comments inside the source code where I believed that they were most helpful, but further work on this would be beneficial.

The implementation of the RDC protocol can also be further enhanced and optimized. The timings can be optimized, as well as the number of symbols needed for the CAD detection. A phase lock system can be added, as well as a "listen before talk" feature.

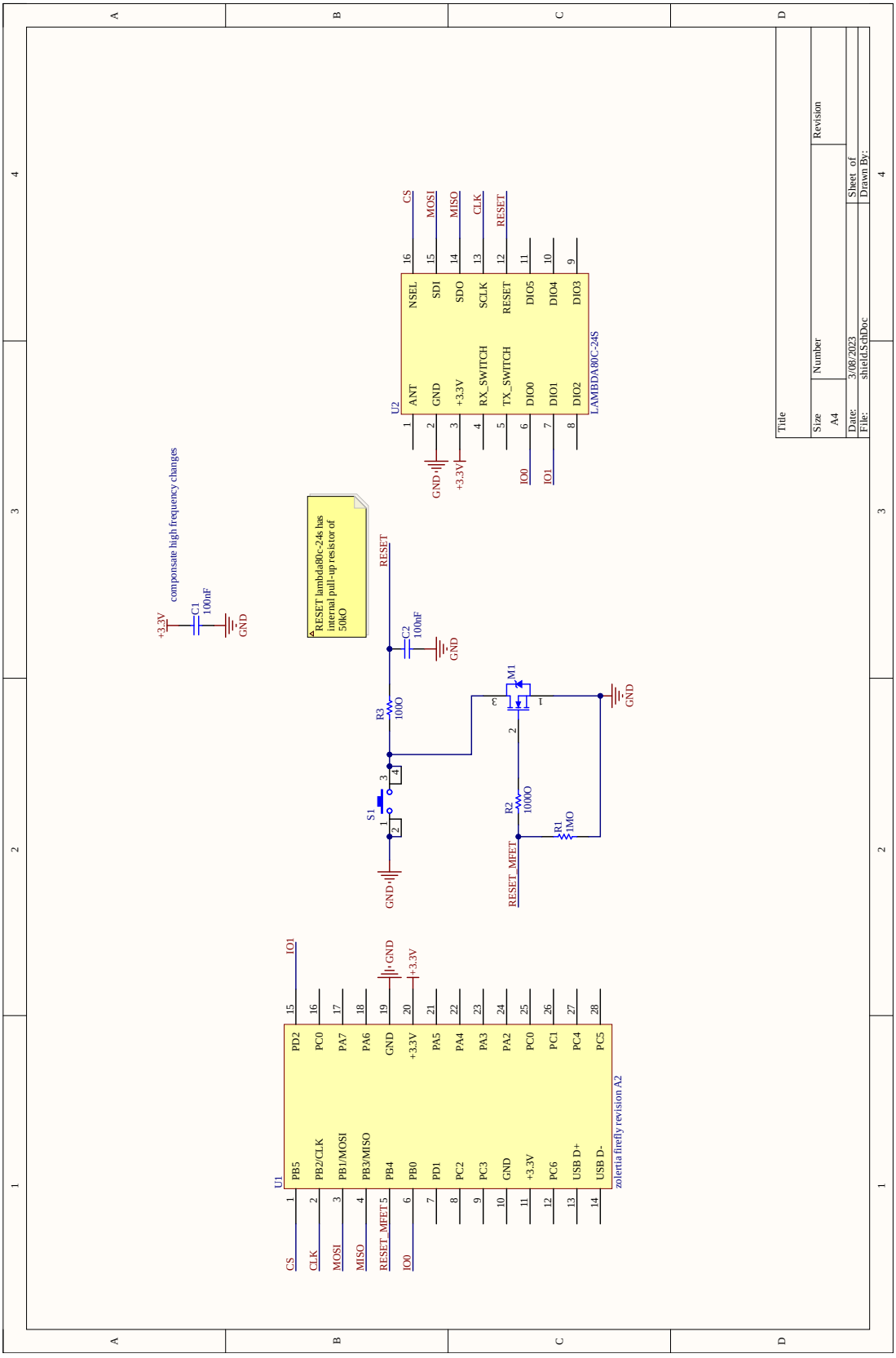
It would be really interesting to add these features and test the protocol with and without them for comparison purposes. In particular the energy-efficiency improvement of a phase lock system and the congestion improvement of a listen before talk feature would be interesting to research.

After all these extra enhancements, the driver would need to be thoroughly tested again and could be included to the MAC layer of Contiki-NG.

Appendix **A**

Appendices

A.1 Schematic of the first radio shield



A.2 Available commands for the LoRa SX1280 transceiver

This table comes from [20].

Command	Opcode	Parameters	Return
GetStatus	0xC0	-	status
WriteRegister	0x18	address[15:8], address[7:0], data[0:n]	-
ReadRegister	0x19	address[15:8], address[7:0]	data[0:n-1]
WriteBuffer	0x1A	offset,data[0:n]	-
ReadBuffer	0x1B	offset	data[0:n-1]
SetSleep	0x84	sleepConfig	-
SetStandby	0x80	standbyConfig	-
SetFs	0xC1	-	-
SetTx	0x83	periodBase, periodBaseCount[15:8], periodBaseCount[7:0]	-
SetRx	0x82	periodBase, periodBaseCount[15:8], periodBaseCount[7:0]	-
SetRxDutyCycle	0x94	rxPeriodBase, rxPeriodBaseCount[15:8], rxPeriodBaseCount[7:0], sleepPeriodBase, sleepPeriodBaseCount[15:8], sleepPeriodBaseCount[7:0]	-
SetCad	0xC5	-	-
SetTxContinuousWave	0xD1	-	-
SetTxContinuousPreamble	0xD2	-	-
SetPacketType	0x8A	packetType	-
GetPacketType	0x03	-	packetType
SetRfFrequency	0x86	rfFrequency[23:16],rfFrequency[15:8], rfFrequency[7:0]	-
SetTxParams	0x8E	power, rampTime	-
SetCadParams	0x88	cadSymbolNum	-
SetBufferBaseAddress	0x8F	txBaseAddress, rxBaseAddress	-
SetModulationParams	0x8B	modParam1, modParam2, modParam3	-

Command	Opcode	Parameters	Return
SetPacketParams	0x8C	packetParam1, packetParam2, packetParam3, packetParam4, packetParam5, packetParam6, packetParam7	-
GetRxBufferStatus	0x17	-	payloadLength, rxBufferOffset
GetPacketStatus	0x1D	-	packetStatus[39:32], packetStatus[31:24], packetStatus[23:16], packetStatus[15:8], packetStatus[7:0]
GetRssiInst	0x1F	-	rssiInst
SetDioIrqParams	0x8D	irqMask[15:8], irqMask[7:0], dio1Mask[15:8],dio1Mask[7:0], dio2Mask[15:8], dio2Mask[7:0], dio3Mask[15:8], dio3Mask[7:0]	-
GetIrqStatus	0x15	-	irqStatus[15:8], irqStatus[7:0]
ClrIrqStatus	0x97	irqMask[15:8], irqMask[7:0]	-
SetRegulatorMode	0x96	regulatorMode	-
SetSaveContext	0xD5	-	-
SetAutoFS	0x9E	0x00: disable or 0x01: enable	-
SetAutoTx	0x98	time	-
SetPerfCounterMode	0x9C	perfCounterMode	-
SetLongPreamble	0x9B	enable	-
SetUartSpeed	0x9D	uartSpeed	-
SetRangingRole	0xA3	0x00=Slave or 0x01=Master	-

Bibliography

[1] C.-o. a 150 contributors. 'The Contiki Operating System'. In: (2018). URL: https://github.com/contiki-os/contiki.

[2] C.-o. a 150 contributors. 'The Contiki Operating System'. In: (2018). URL: https://docs.contiki-ng.org/en/develop/.

[3] F. Adelantado, X. Vilajosana, P. Tuset-Peiro, B. Martinez, J. Melia-Segui, and T. Watteyne. 'Understanding the Limits of LoRaWAN'. In: *IEEE Communications Magazine* 55.9 (2017), pp. 34–40. DOI: 10.1109/MCOM.2017.1600613.

[4] contributors Contiki-NG. *LoRa modulation basics*. 2022. URL: https://docs.contiki-ng.org/en/develop/doc/platforms/zolertia/Zolertia-Firefly-Revision-A.html (visited on 08/13/2023).

[5] C.-o. contributors. *Radio duty cycling*. 2019. URL: https://github.com/contiki-os/contiki/wiki/Radio-duty-cycling (visited on 08/13/2023).

[6] J. R. Cotrim and J. H. Kleinschmidt. 'LoRaWAN Mesh Networks: A Review and Classification of Multihop Communication'. In: *Sensors* 20.15 (2020). ISSN: 1424-8220. DOI: 10.3390/s20154273. URL: https://www.mdpi.com/1424-8220/20/15/4273.

[7] A. Dunkels. 'The ContikiMAC radio duty cycling protocol'. In: (May 2012).

[8] V. Electric. *How LoRa Modulation really works - long range communication using chirps*. 2021. URL: https://www.youtube.com/watch?v=jHWepP1ZWTk (visited on 08/13/2023).

[9] D. Huang. *Wi-Fi, LoRaWAN, and IoT convergence*. 2023. URL: https://wballiance.com/guest-blog-wi-fi-lorawan-and-iot-convergence/ (visited on 08/13/2023).

[10] T. Janssen, N. BniLam, M. Aernouts, R. Berkvens, and M. Weyn. 'LoRa 2.4 GHz Communication Link and Range'. In: *Sensors* 20.16 (2020). ISSN: 1424-8220. DOI: 10.3390/s20164366. URL: https://www.mdpi.com/1424-8220/20/16/4366.

[11] R. Lacoste. *DSSS in a nutchell*. 2020. URL: https://circuitcellar.com/research-design-hub/dsss-in-a-nutshell/ (visited on 08/13/2023).

[12] M. Michel and B. Quoitin. 'Technical Report : ContikiMAC vs X-MAC performance analysis'. In: (Apr. 2014). URL: https://www.researchgate.net/publication/261636095_Technical_Report_ContikiMAC_vs_X-MAC_performance_analysis.

[13] G. Oikonomou, S. Duquennoy, A. Elsts, J. Eriksson, Y. Tanaka, and N. Tsiftes. 'The Contiki-NG open source operating system for next generation IoT devices'. In: *SoftwareX* 18 (2022), p. 101089. ISSN: 2352-7110. DOI: https://doi.org/10.1016/j.softx.2022.101089.

[14] T. Perale. *SX128X LoRa transceiver driver for contiki-ng OS*. 2022. URL: https://github.com/tperale/sx128x (visited on 04/17/2023).

[15] B. Sartori, S. Thielemans, M. Bezunartea, A. Braeken, and K. Steenhaut. 'Enabling RPL multihop communications based on LoRa'. In: (2017), pp. 1–8. DOI: 10.1109/WiMOB.2017.8115756.

[16] Semtech. *LoRa modulation basics*. 2016. URL: https://www.frugalprototype.com/wp-content/uploads/2016/08/an1200.22.pdf (visited on 08/13/2023).

[17] Semtech. *What are LoRa and LoRaWAN*. 2023. URL: https://www.semtech.com/lora/what-is-lora (visited on 08/13/2023).

[18] Semtech. *What is LoRa*. 2023. URL: https://lora-developers.semtech.com/documentation/tech-papers-and-guides/lora-and-lorawan/ (visited on 08/13/2023).

[19] Semtech. *Channel Activity Detection: Ensuring Your LoRa® Packets are sent*. URL: https://lora-developers.semtech.com/documentation/tech-papers-and-guides/channel-activity-detection-ensuring-your-lora-packets-are-sent/how-to-ensure-your-lora-packets-are-sent-properly/ (visited on 08/13/2023).

[20] Semtech. *SX1280-1-V3.2*. URL: https://semtech.my.salesforce.com/sfc/p/#E0000000JelG/a/2R000000HoCW/8EVYKPLcthcKCB_cKzApAc6Xf6tAHtn9.UKcOh7SNmg (visited on 08/13/2023).

[21] R. solutions. *LAMBDA80 2.4GHz LoRa Transceiver*. 2020. URL: https://www.rfsolutions.co.uk/downloads/6245d8c44da69987DS-LAMBDA80-1.pdf (visited on 08/13/2023).

[22] S. Support. *Logic Interferes with My Circuit Operation*. 2022. URL: https://support.saleae.com/troubleshooting/logic-interferes-with-circuit (visited on 08/13/2023).

[23] M.-P. Uwase, M. Bezunartea, J. Tiberghien, J.-M. Dricot, and K. Steenhaut. 'Experimental Comparison of Radio Duty Cycling Protocols for Wireless Sensor Networks'. In: *IEEE Sensors Journal* 17.19 (2017), pp. 6474–6482. DOI: 10.1109/JSEN.2017.2738700.

[24] L. Vangelista. 'Frequency Shift Chirp Modulation: The LoRa Modulation'. In: *IEEE Signal Processing Letters* 24.12 (2017), pp. 1818–1821. DOI: 10.1109/LSP.2017.2762960.